驱动使用方法

目录

驱动使用方法 1
产品对应的驱动类型 3
一、LpmsIG1OpenSourceLib 4
1.编译 Library 4
2 编译 linux 示例程序 5
3. ROS1 7
4.ROS2 14
二、 Openzen 20
1. Windows 有线连接 20
2. windows 蓝牙无线连接（B2） 23
3. Linux 有线连接 24
4.Linux 蓝牙无线连接（B2） 25
三、 Openzenros 28
1.编译 Library 28
2.运行 ROS 节点 29
3 已发布的主题(rostopic list 可查看) 30
4 服务（rosservice list 可查看） 31
四、 Openzenros2（foxy 版本为示例） 32
1.编译 Library 32
2.运行 ROS2 节点 33
3 已发布的主题(ros2 topic list 可查看) 34
4 服务（ros2 service list 可查看） 34
5 Humble 版本编译遇到问题有两个解决办法 35
五、 Lpmsnav2sopensourcelib 38
1. 编译 Library 38
2 编译 linux 示例程序 39
3.编译 ROS 示例程序 41
4.运行 ROS 节点 42
5.已发布的主题（rostopic list 可查看） 43
6.服务（rosservice list 可查看） 43
六、 LpSensorPy 44
windows 蓝牙无线连接（B2） 44
七、 常见问题 47
1 驱动编译和运行 47
2 驱动输出的数值 56

产品对应的驱动类型

名字	系统	传感器类型
LpmsIG1opensourcelib(ROS1/2) OpenzenRos(ROS1) OpenzenRos2(ROS2) Openzen	Linux	LPMS-IG1 系列、 LPMS-BE 系列、 LPMS-NAV3 系列、 LPMS-SI 系列、 LPMS-CURS 系列、 LPMS-AL3 系列、 LPMS-U3 系列、
LpmsNAV2OpenSourceLib(ROS1)	Linux	LPMS-NAV2 系列 LPMS-NAV2S 系列
Openzen/LpSensorPy	Windows/Linux	LPMS-B2 系列

一、LpmsIG1OpenSourceLib

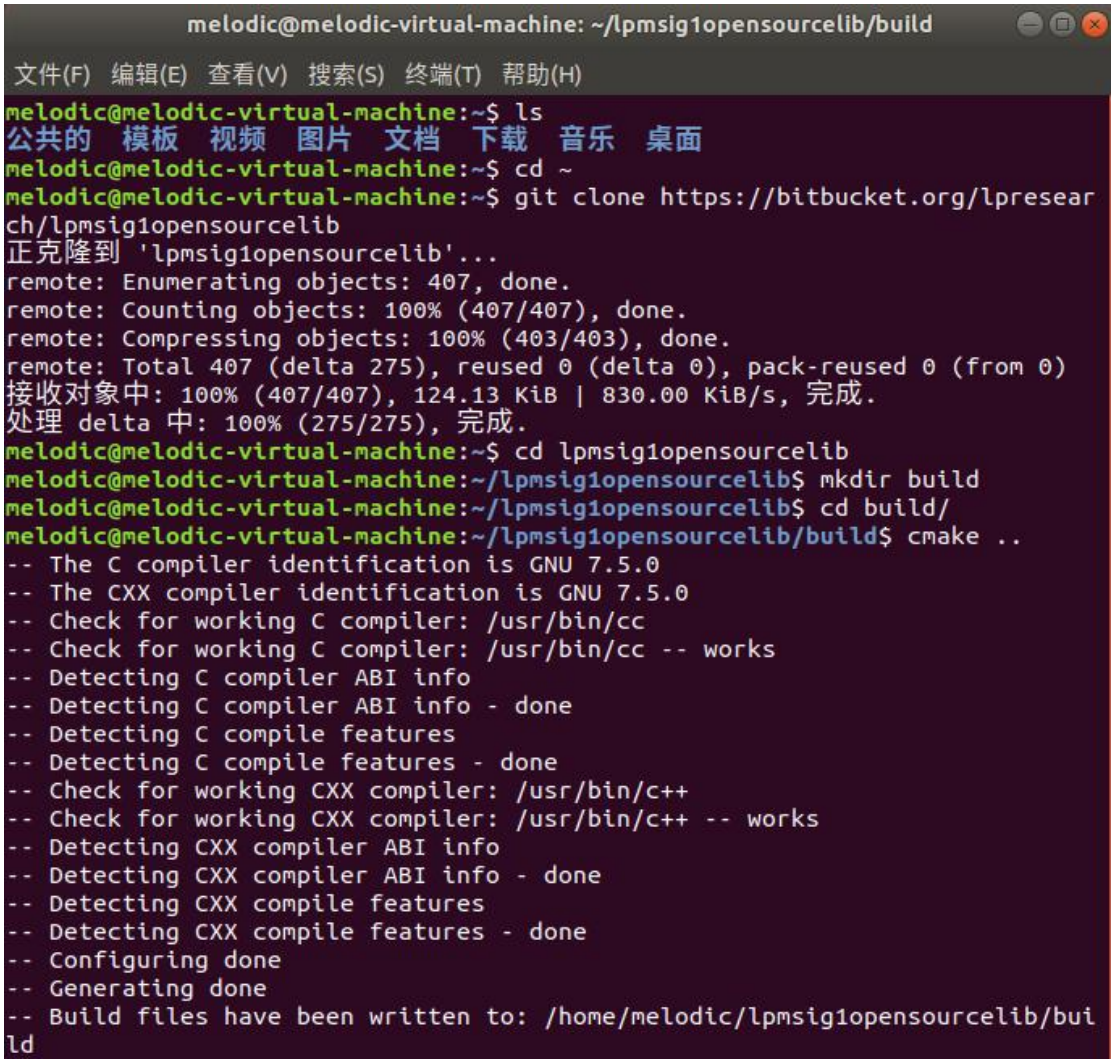
驱动开源网站：[lpresearch / lpmsig1opensourcelib — Bitbucket]

1.编译 Library

1.1 步骤

```
$ cd ~
$ git clone https://bitbucket.org/lpresearch/lpmsig1opensourcelib
$ cd lpmsig1opensourcelib
$ mkdir build
$ cd build
$ cmake ..
$ make
$ make package
$ sudo dpkg -i libLpmsIG1_OpenSource-x.y.z-Linux.deb
```

1.2 示例图片



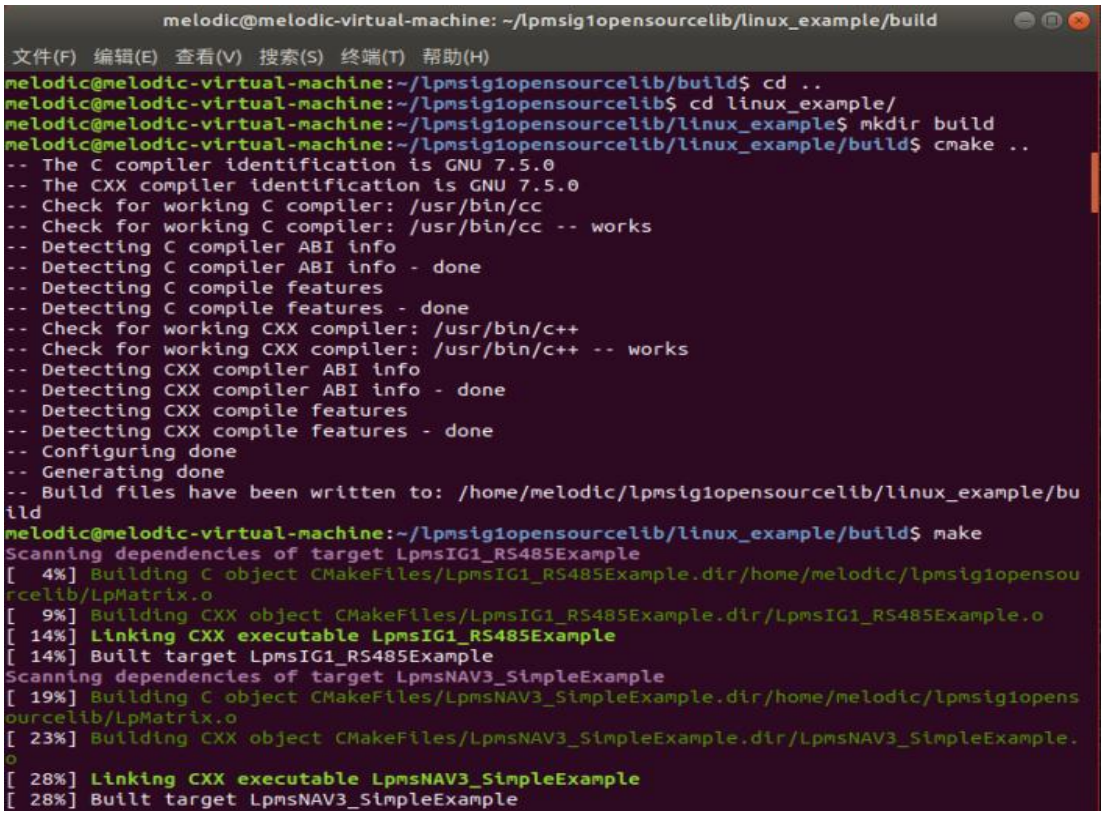
```
melodic@melodic-virtual-machine:~/lpmsig1opensourcelib/build$ make
Scanning dependencies of target LpmsIG1_OpenSourceLib
[ 16%] Building CXX object CMakeFiles/LpmsIG1_OpenSourceLib.dir/MicroMeasur
e.o
[ 33%] Building CXX object CMakeFiles/LpmsIG1_OpenSourceLib.dir/SerialPortL
inux.o
[ 50%] Building CXX object CMakeFiles/LpmsIG1_OpenSourceLib.dir/LpmsIG1.o
[ 66%] Building C object CMakeFiles/LpmsIG1_OpenSourceLib.dir/LpMatrix.o
[ 83%] Building CXX object CMakeFiles/LpmsIG1_OpenSourceLib.dir/LpUtil.o
[100%] Linking CXX shared library libLpmsIG1_OpenSourceLib.so
[100%] Built target LpmsIG1_OpenSourceLib
melodic@melodic-virtual-machine:~/lpmsig1opensourcelib/build$ make package
[100%] Built target LpmsIG1_OpenSourceLib
Run CPack packaging tool...
CPack: Create package using DEB
CPack: Install projects
CPack: - Run preinstall target for: LpmsIG1_OpenSourceLib
CPack: - Install project: LpmsIG1_OpenSourceLib
CPack: Create package
CPack: - package: /home/melodic/lpmsig1opensourcelib/build/libLpmsIG1_OpenS
ource-0.3.3-Linux.deb generated.
melodic@melodic-virtual-machine:~/lpmsig1opensourcelib/build$ sudo dpkg -i
libLpmsIG1_OpenSource-0.3.3-Linux.deb
[sudo] melodic 的密码:
(正在读取数据库 ... 系统当前共安装有 236497 个文件和目录。)
正准备解包 libLpmsIG1_OpenSource-0.3.3-Linux.deb  ...
正在将 liblpmsig1_opensource (0.3.3) 解包到 (0.3.3) 上 ...
正在设置 liblpmsig1_opensource (0.3.3) ...
```

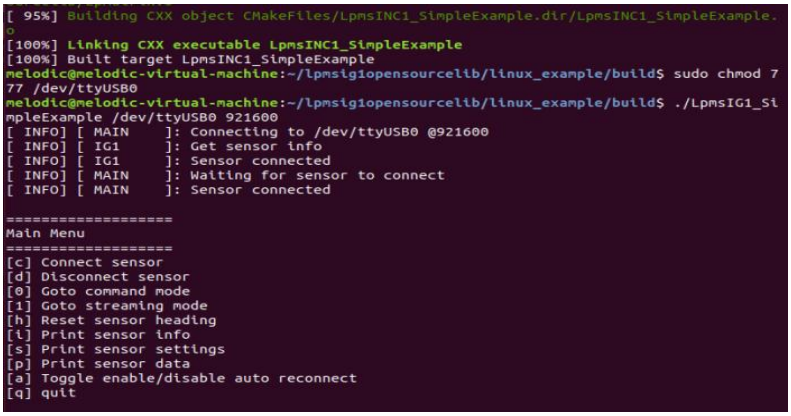
2 编译 linux 示例程序

2.1 步骤

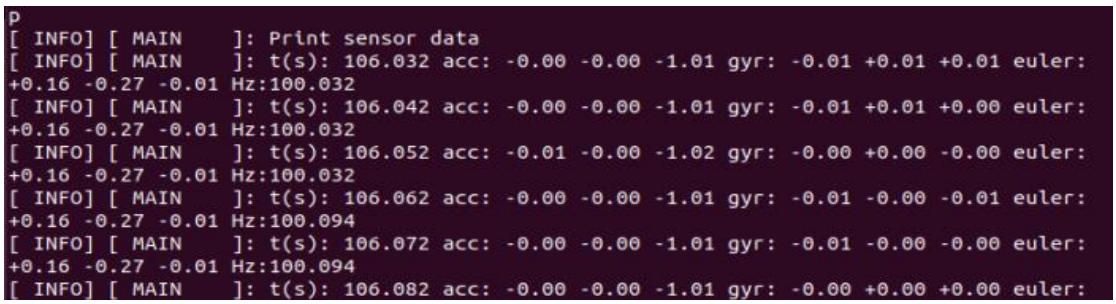
```
    $ cd linux_example
    $ mkdir build
    $ cd build
    $ cmake ..
    $ make
# 给端口权限(可以通过dmesg | grep tty之类的指令查看端口，示例中是USB0实际上可能是其他的)
    $ sudo chmod 777 /dev/ttyUSB0
    $ ./LpmsIG1_SimpleExample /dev/ttyUSB0 921600
```

2.2 示例图片





2.3 运行结果



3. ROS1

3.1 编译 ROS1 示例程序

3.1.1 步骤

```
# 创建ROS工作区.
    $ mkdir -p ~/catkin_ws/src
    $ cd ~/catkin_ws/src

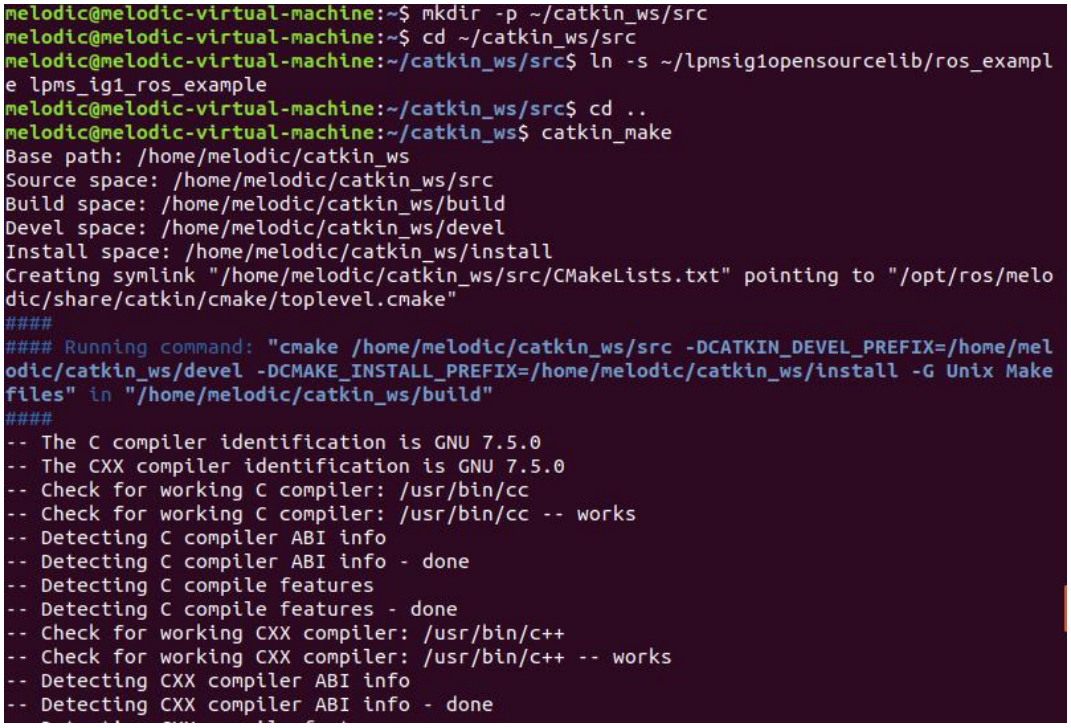
# 我们建议创建一个指向ros_example文件夹的符号链接，而不是复制
    $ ln -s ~/lpmsig1opensourcelib/ros_example lpms_ig1_ros_example

# 编译ROS示例程序
    $ cd ~/catkin_ws
    $ catkin_make
    $ source ./devel/setup.bash

# 给端口权限(可以通过dmesg | grep tty之类的指令查看端口，示例中是USB0实际上可能是其他的)
    $ sudo chmod 777 /dev/ttyUSB0
```

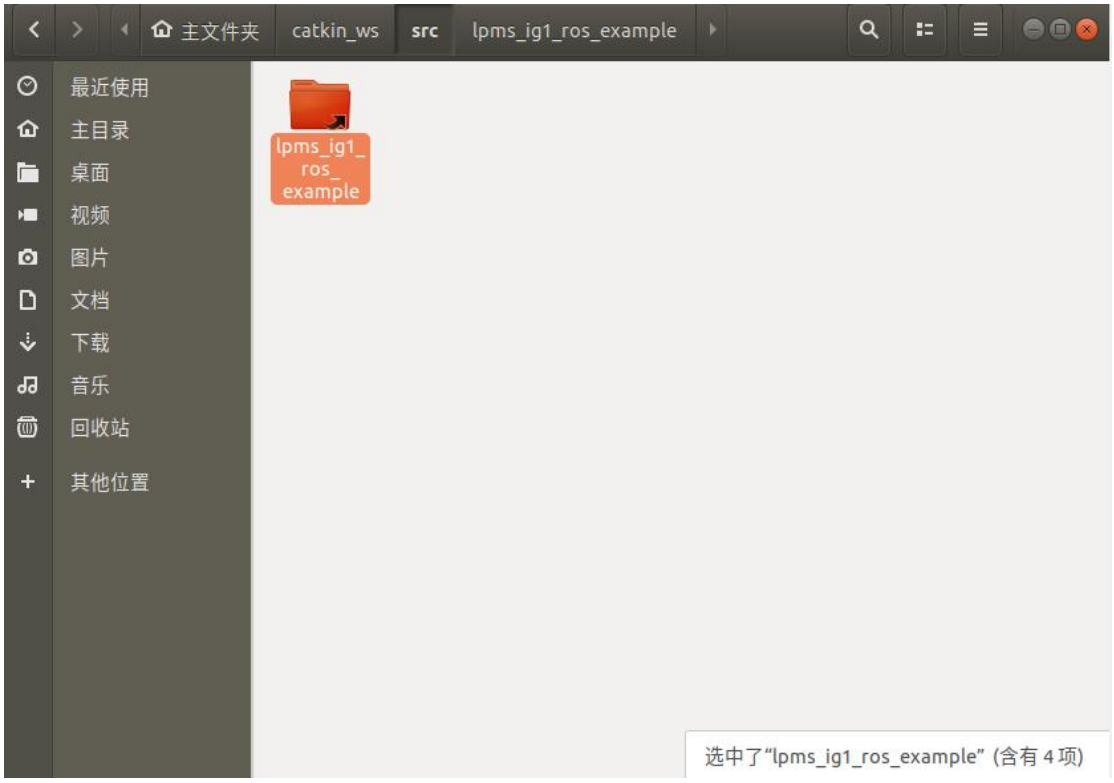
如果出现报错可以查看下方的报错排查

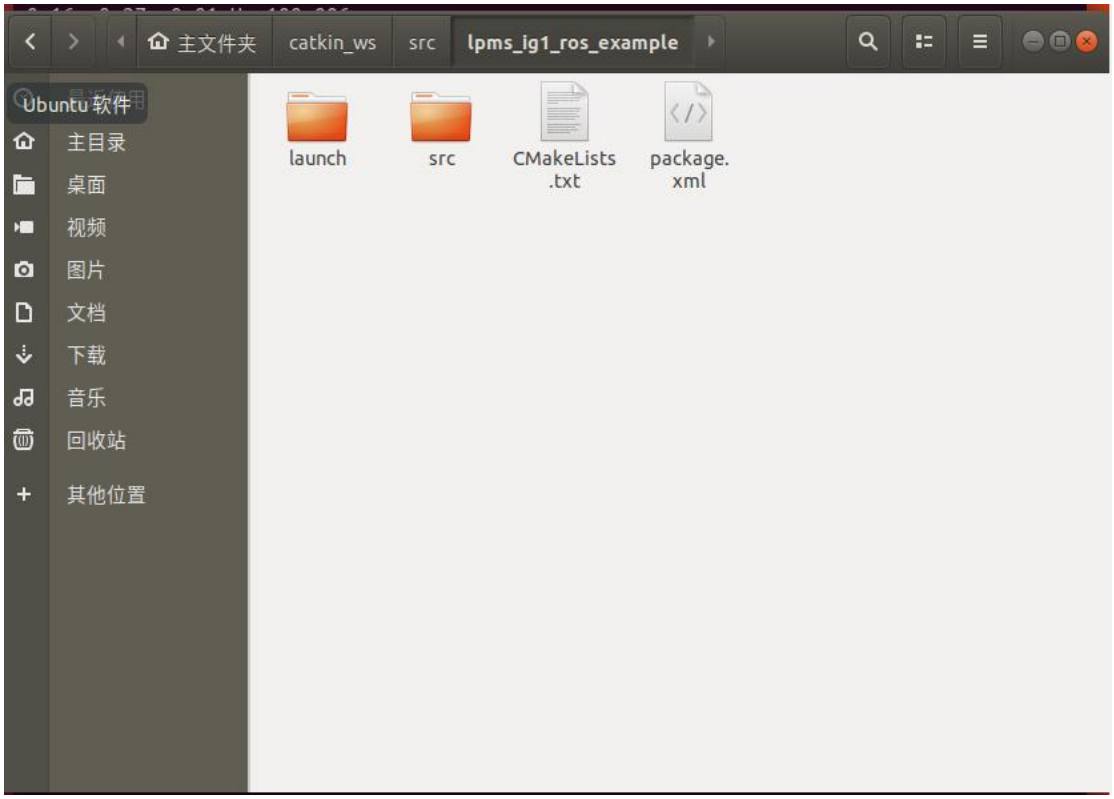
3.1.2 示例图片



```
Scanning dependencies of target lpms_nav3_node
[ 52%] Building CXX object lpms_ig1_ros_example/CMakeFiles/lpms_nav3_node.dir/src/lpms_nav3_node.cpp.o
[ 58%] Linking CXX executable /home/melodic/catkin_ws/devel/lib/lpms_ig1/lpms_ig1_rs485_node
[ 58%] Built target lpms_ig1_rs485_node
Scanning dependencies of target lpms_ig1_node
[ 64%] Building CXX object lpms_ig1_ros_example/CMakeFiles/lpms_ig1_node.dir/src/lpms_ig1_node.cpp.o
[ 70%] Linking CXX executable /home/melodic/catkin_ws/devel/lib/lpms_ig1/lpms_nav3_node
[ 70%] Built target lpms_nav3_node
Scanning dependencies of target lpms_si1_node
[ 76%] Building CXX object lpms_ig1_ros_example/CMakeFiles/lpms_si1_node.dir/src/lpms_si1_node.cpp.o
[ 82%] Linking CXX executable /home/melodic/catkin_ws/devel/lib/lpms_ig1/lpms_ig1_node
[ 82%] Built target lpms_ig1_node
Scanning dependencies of target imudata_rad_to_deg_node
[ 88%] Building CXX object lpms_ig1_ros_example/CMakeFiles/imudata_rad_to_deg_node.dir/src/imudata_rad_to_deg_node.cpp.o
[ 94%] Linking CXX executable /home/melodic/catkin_ws/devel/lib/lpms_ig1/lpms_si1_node
[ 94%] Built target lpms_si1_node
Scanning dependencies of target lpms_ig1_generate_messages
[ 94%] Built target lpms_ig1_generate_messages
[100%] Linking CXX executable /home/melodic/catkin_ws/devel/lib/lpms_ig1/imudata_rad_to_deg_node
[100%] Built target imudata_rad_to_deg_node
melodic@melodic-virtual-machine:~/catkin_ws$ source ./devel/setup.bash
melodic@melodic-virtual-machine:~/catkin_ws$ sudo chmod 777 /dev/ttyUSB0
```

进行完软连接后 Src 文件内容如下，请确认清楚：





3.2 运行 ROS 节点

3.2.1 步骤

打开一个新的终端窗口并运行roscore

```
$ roscore
```

现在，您可以在其他终端窗口上运行lpms_ig1节点。 以下是在 lpms_ig1 节点启动 ros 的一些示例命令：

IG1系列（注意示例中端口是以ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
$ rosrun lpms_ig1 lpms_ig1_node _port:=/dev/ttyUSB0 _baudrate:=921600
```

IG1-RS485（注意示例中端口是以ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
$ rosrun lpms_ig1 lpms_ig1_rs485_node _port:=/dev/ttyUSB0  _baudrate:=115200
```

BE系列（注意示例中端口是以ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
$ rosrun lpms_ig1 lpms_be1_node _port:=/dev/ttyUSB0 _baudrate:=115200
```

NAV3系列（注意示例中端口是以ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
$ rosrun lpms_ig1 lpms_nav3_node _port:=/dev/ttyUSB0 _baudrate:=115200
```

U系列/CURS系列/SI系列/AL系列（注意示例中端口是以ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
$ rosrun lpms_ig1 lpms_curs3_node _port:=/dev/ttyUSB0 _baudrate:=921600
```

3.2.2 示例图片

```
roscore http://melodic-virtual-machine:11311/
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
melodic@melodic-virtual-machine:~$ roscore
... logging to /home/melodic/.ros/log/cf2a2efa-b76b-11ef-8e51-000c2937bb16/roslaun
ch-melodic-virtual-machine-4110.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://melodic-virtual-machine:37455/
ros_comm version 1.14.13


SUMMARY
========

PARAMETERS
 * /rosdistro: melodic
 * /rosversion: 1.14.13

NODES

auto-starting new master
process[master]: started with pid [4125]
ROS_MASTER_URI=http://melodic-virtual-machine:11311/

setting /run_id to cf2a2efa-b76b-11ef-8e51-000c2937bb16
process[rosout-1]: started with pid [4136]
started core service [/rosout]
```

3.2.3 运行结果

```
melodic@melodic-virtual-machine:~/catkin_ws$ ^C
melodic@melodic-virtual-machine:~/catkin_ws$ rosrun lpms_ig1 lpms_ig1_node _port:=/dev/t
tyUSB0 _baudrate:=921600
[ INFO] [1733886098.816401193]: Settings
[ INFO] [1733886098.816978308]: Port: /dev/ttyUSB0
[ INFO] [1733886098.817066515]: Baudrate: 921600
[ INFO] [1733886098.817129739]: Auto reconnect: Enabled
[ INFO] [ IG1     ]: Get sensor info
[ INFO] [ IG1     ]: Sensor connected
[ INFO] [1733886099.020750099]: Waiting for sensor to connect 2
[ INFO] [1733886100.021207548]: Sensor connected
```

此时可以 rostopic echo /imu/data 打印数据输出

```
melodic@melodic-virtual-machine: ~
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
header:
  seq: 4600
  stamp:
    secs: 1733885985
    nsecs: 596551296
  frame_id: "imu"
orientation:
  x: 0.00129484117497
  y: -0.00227217026986
  z: -5.49382093595e-05
  w: 0.999996602535
orientation_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
angular_velocity:
  x: -0.000145769787506
  y: 0.000158905426243
  z: -0.000179342050114
angular_velocity_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
linear_acceleration:
  x: 0.0373284602724
  y: 0.00984458925901
  z: 9.94195645452
linear_acceleration_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
---
```

3.3 运行 launch 文件

3.3.1 步骤（运行 launch 文件之前麻烦请先运行 node 节点看是否能连接上）

IG1系列（注意launch文件中的代码端口是以/dev/ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
roslaunch lpms_ig1 lpmsig1.launch
```

IG1-RS485（注意launch文件中的代码端口是以/dev/ttyTHS5为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
roslaunch lpms_ig1 lpmsig1_rs485.launch
```

BE系列（注意launch文件中的代码端口是以0001为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
roslaunch lpms_ig1 lpmsbe1.launch
```

NAV3系列（注意launch文件中的代码端口是以/dev/ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

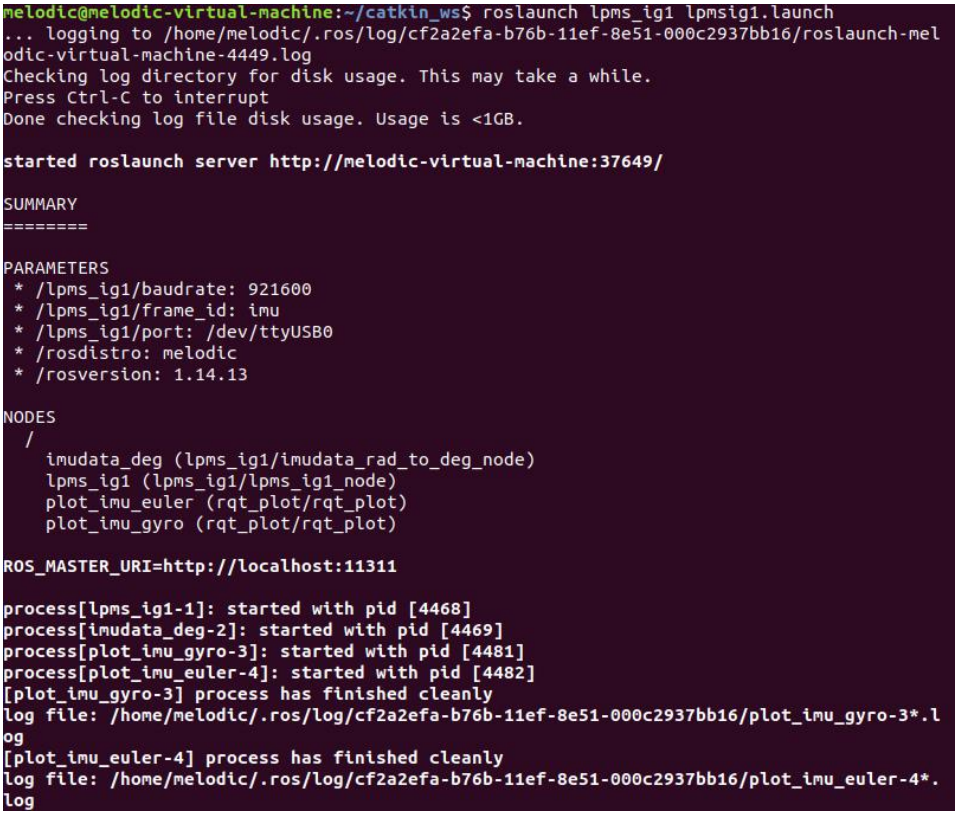
```
roslaunch lpms_ig1 lpmsnav3.launch
```

U系列/CURS系列/SI系列/AL系列（注意launch文件中的代码端口是以/dev/ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

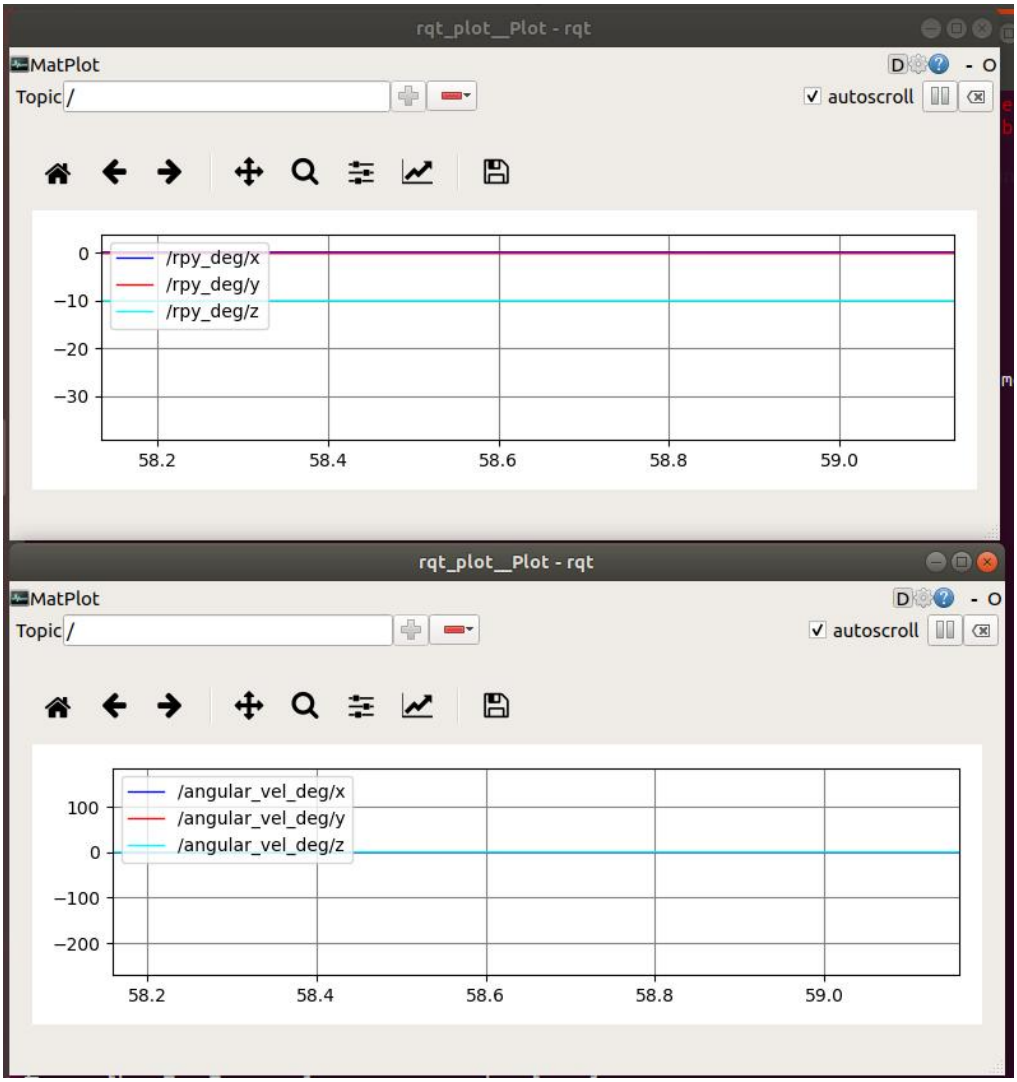
```
roslaunch lpms_ig1 lpmscurs3.launch
```

如果出现launch文件出现报错无法启动的时候，请先测试启动node节点是否正常

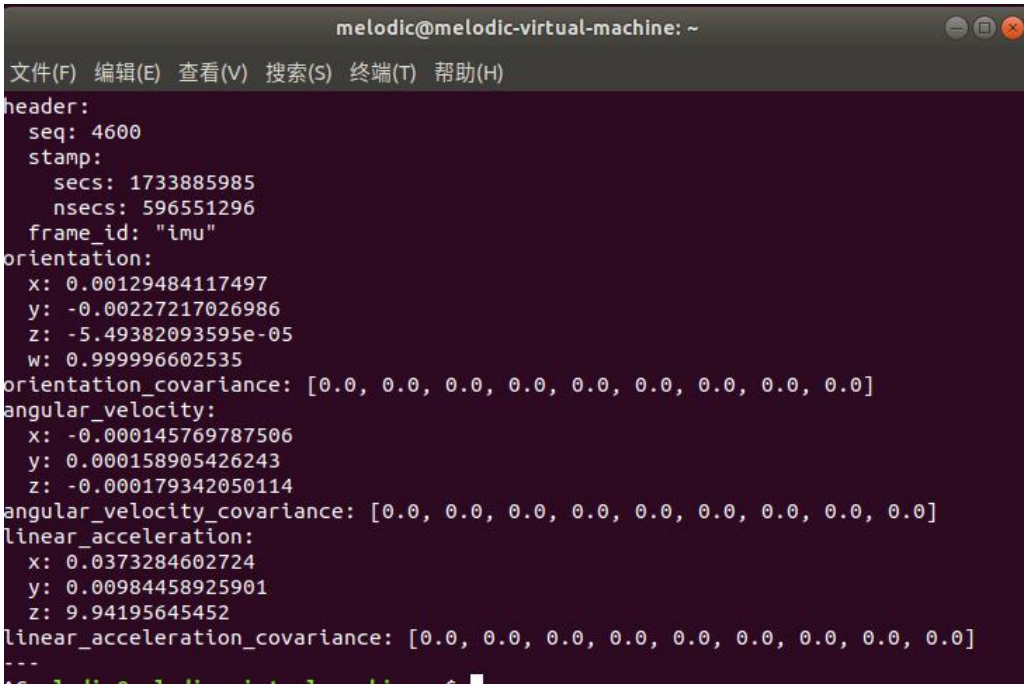
3.3.2 示例图片



3.3.3 弹出窗口输出数据

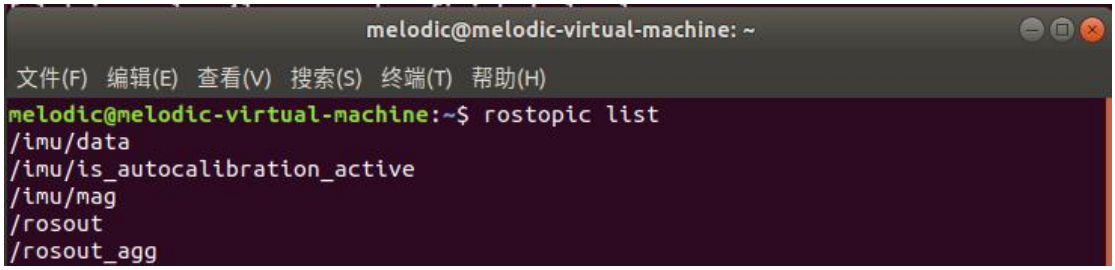


也可以 rostopic echo /imu/data 打印数据输出



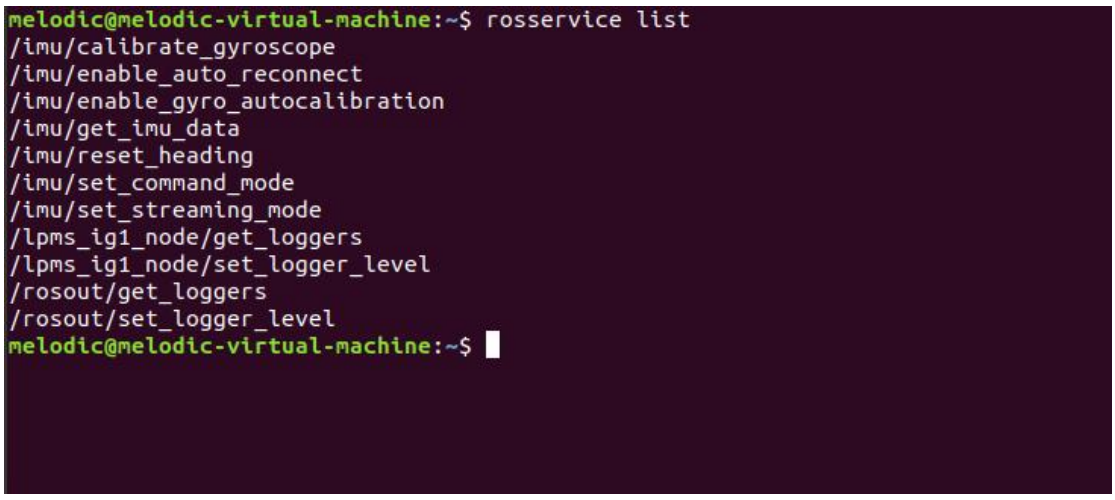
3.4 已发布的主题(rostopic list 可查看)

3.4.1 示例图片



3.5 服务（rosservice list 可查看）

3.5.1 示例图片



3.5.2 常用服务节点解释

航向角归零

```
$ rosservice call /imu/reset_heading
```

陀螺仪手动校准

```
$ rosservice call /imu/calibrate_gyroscope
```

启开陀螺仪自动校准

```
$ rosservice call /imu/enable_gyro_autocalibration true
```

屏蔽陀螺仪自动校准

```
$ rosservice call /imu/enable_gyro_autocalibration false
```

用户可透过/imu/is_autocalibration_active Topic 来判断陀螺仪自动校准的状态

```
$ rostopic echo /imu/is_autocalibration_active
```

4. ROS2

4.1 编译 ROS2 示例程序

注意：由于前面已经编译了库，这里就不再展示编译库的步骤可以按照第一步来进行（不可缺少）

4.1.1 步骤

```
# 创建ROS工作区.
    $ mkdir -p ros_ws/src
    $ cd ros_ws/src

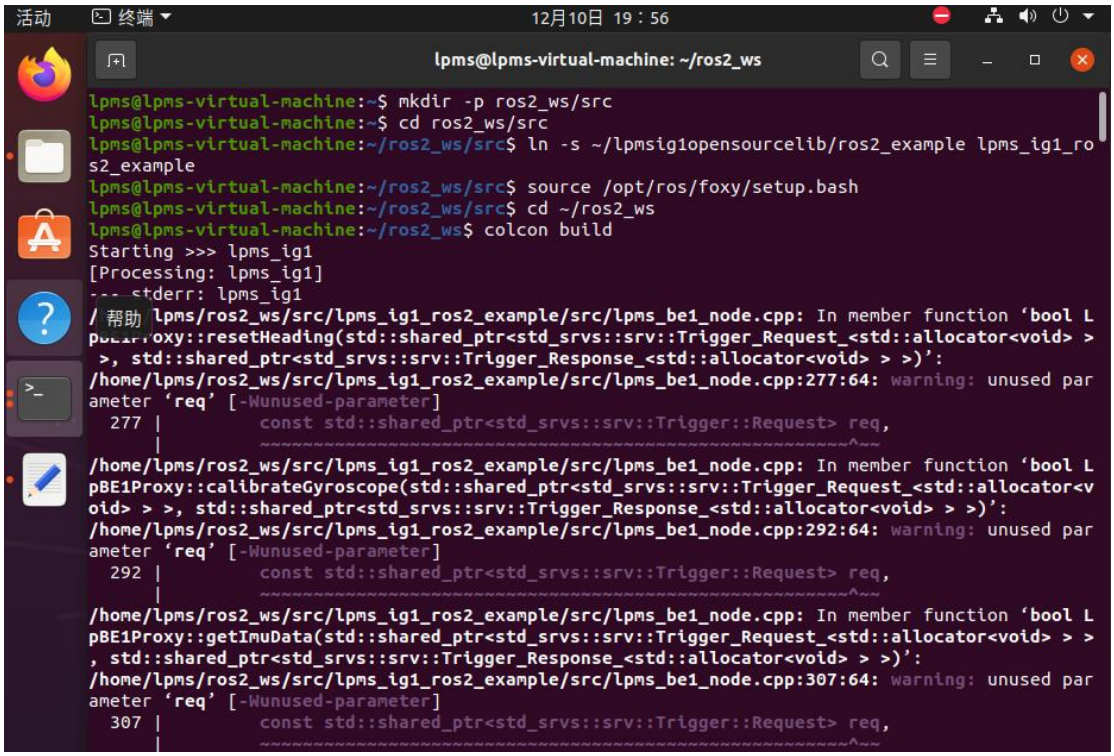
# 我们建议创建一个指向ros_example文件夹的符号链接，而不是复制
    $ ln -s ~/lpmsig1opensourcelib/ros2_example lpms_ig1_ros2_example

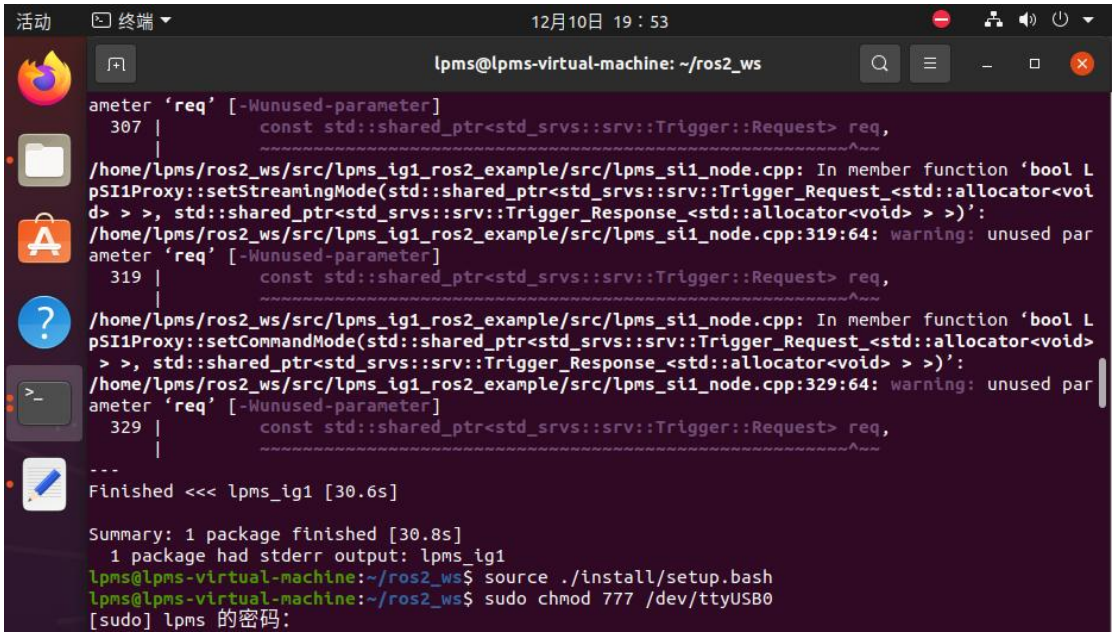
# 刷新
    $ source /opt/ros/foxy/setup.bash

# 编译ROS示例程序
    $ cd ~/ros_ws
    $ colcon build
    $ source ./install/setup.bash

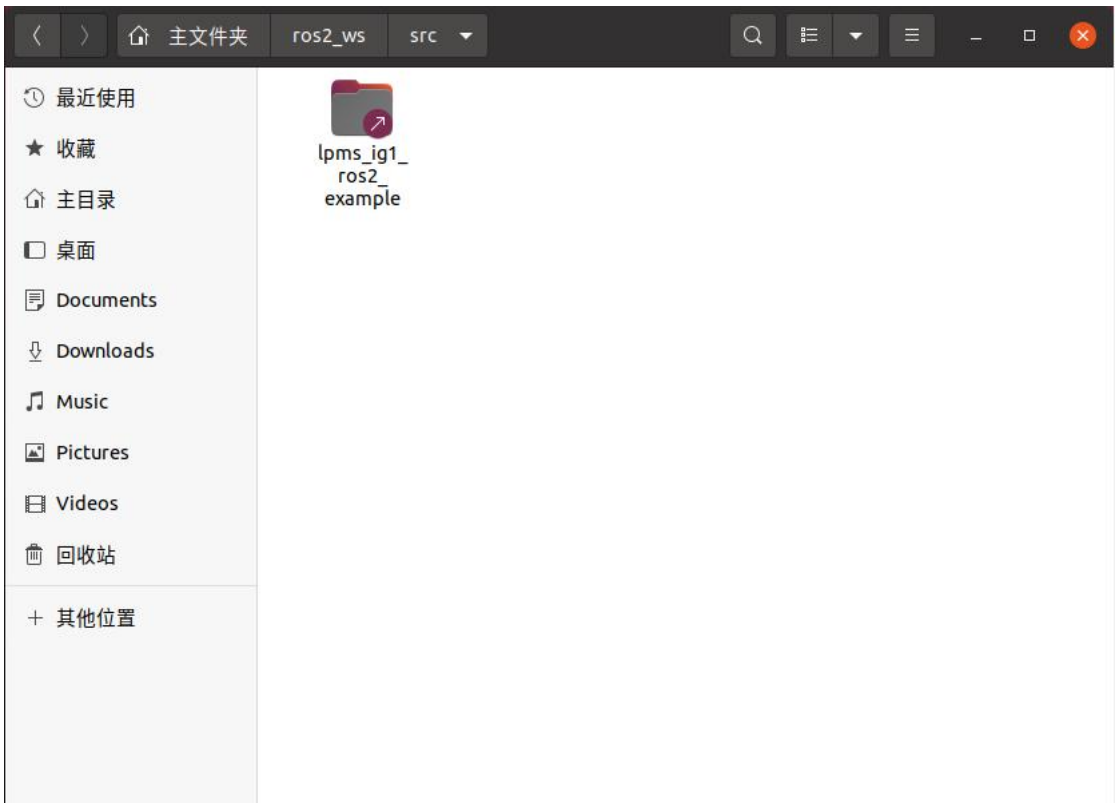
# 给端口权限(可以通过dmesg | grep tty之类的指令查看端口，示例中是USB0实际上可能是其他的)
    $ sudo chmod 777 /dev/ttyUSB0
```

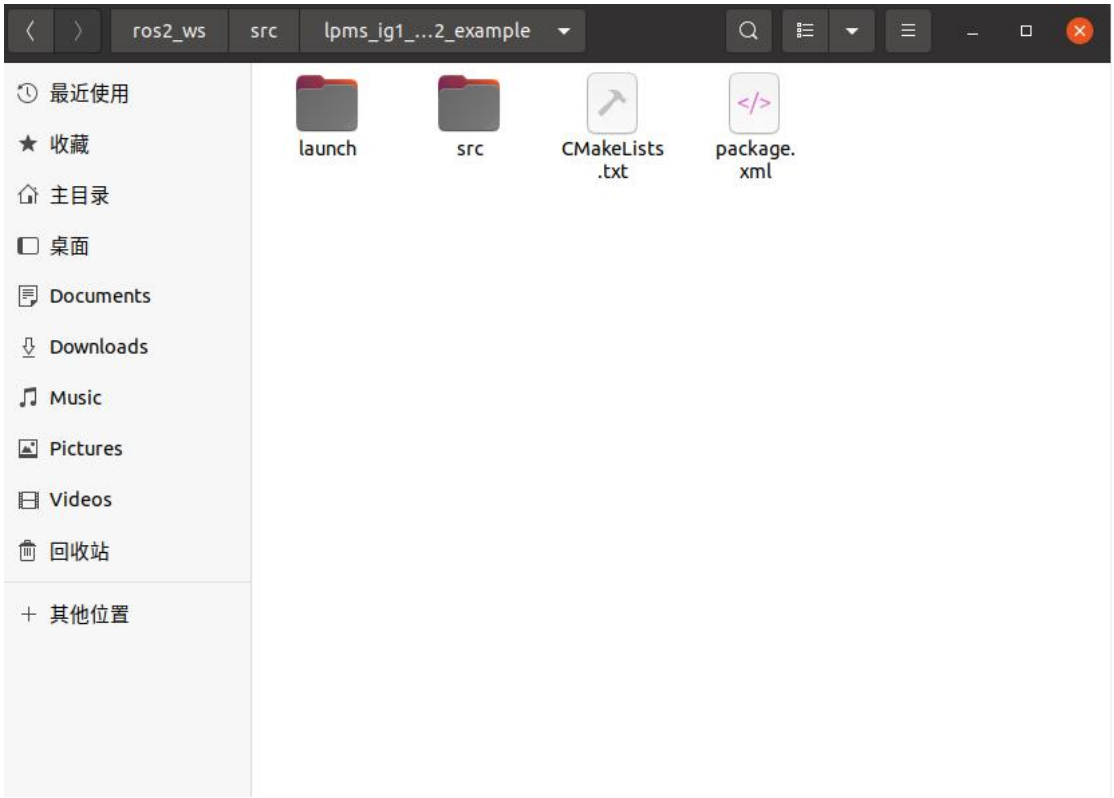
4.1.2 示例图片





进行完软连接后 Src 文件内容如下，请确认清楚：





4.2 运行 ROS2 节点

4.2.1 步骤

IG1系列*（注意示例中端口是以ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
$ ros2 run lpms_ig1 lpms_ig1_node --ros-args --param port:=/dev/ttyUSB0 --param baudrate:=921600
```

IG1-RS485（注意示例中端口是以ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
$ ros2 run lpms_ig1 lpms_ig1_rs485_node --ros-args --param port:=/dev/ttyUSB0 --param baudrate:=115200
```

BE系列（注意示例中端口是以ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
$ ros2 run lpms_ig1 lpms_be1_node --ros-args --param port:=/dev/ttyUSB0 --param baudrate:=115200
```

NAV3系列（注意示例中端口是以ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
$ ros2 run lpms_ig1 lpms_nav3_node --ros-args --param port:=/dev/ttyUSB0 --param baudrate:=115200
```

U系列/CURS系列/SI系列/AL系列（注意示例中端口是以ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
$ ros2 run lpms_ig1 lpms_curs3_node --ros-args --param port:=/dev/ttyUSB0 --param baudrate:=921600
```

4.2.2 示例图片

```
lpms@lpms-virtual-machine:~/ros2_ws$ ^C
lpms@lpms-virtual-machine:~/ros2_ws$ ros2 run lpms_ig1 lpms_ig1_node --ros-args --param port:=
/dev/ttyUSB0 --param baudrate:=921600
[INFO] [1733888572.531428514] [lpms_ig1_node]: Settings
[INFO] [1733888572.531509502] [lpms_ig1_node]: Port: /dev/ttyUSB0
[INFO] [1733888572.531516932] [lpms_ig1_node]: Baudrate: 921600
[INFO] [1733888572.531521267] [lpms_ig1_node]: Auto reconnect: Enabled
[ INFO] [ IG1     ]: Get sensor info
[ INFO] [ IG1     ]: Sensor connected
[INFO] [1733888572.733850524] [lpms_ig1_node]: Waiting for sensor to connect 2
[INFO] [1733888573.734567628] [lpms_ig1_node]: Sensor connected
```

4.3 运行 launch 节点

4.3.1 步骤（运行 launch 文件之前麻烦请先运行 node 节点看是否能连接上）

IG1.系列（注意launch文件中的代码端口是以/dev/ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
ros2 launch lpms_ig1 lpms_ig1_launch.py
```

IG1-RS485（注意launch文件中的代码端口是以/dev/ttyTHS5为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
ros2 launch lpms_ig1 lpms_ig1_rs485_launch.py
```

BE.系列（注意launch文件中的代码端口是以0001为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
ros2 launch lpms_ig1 lpms_be1_launch.py
```

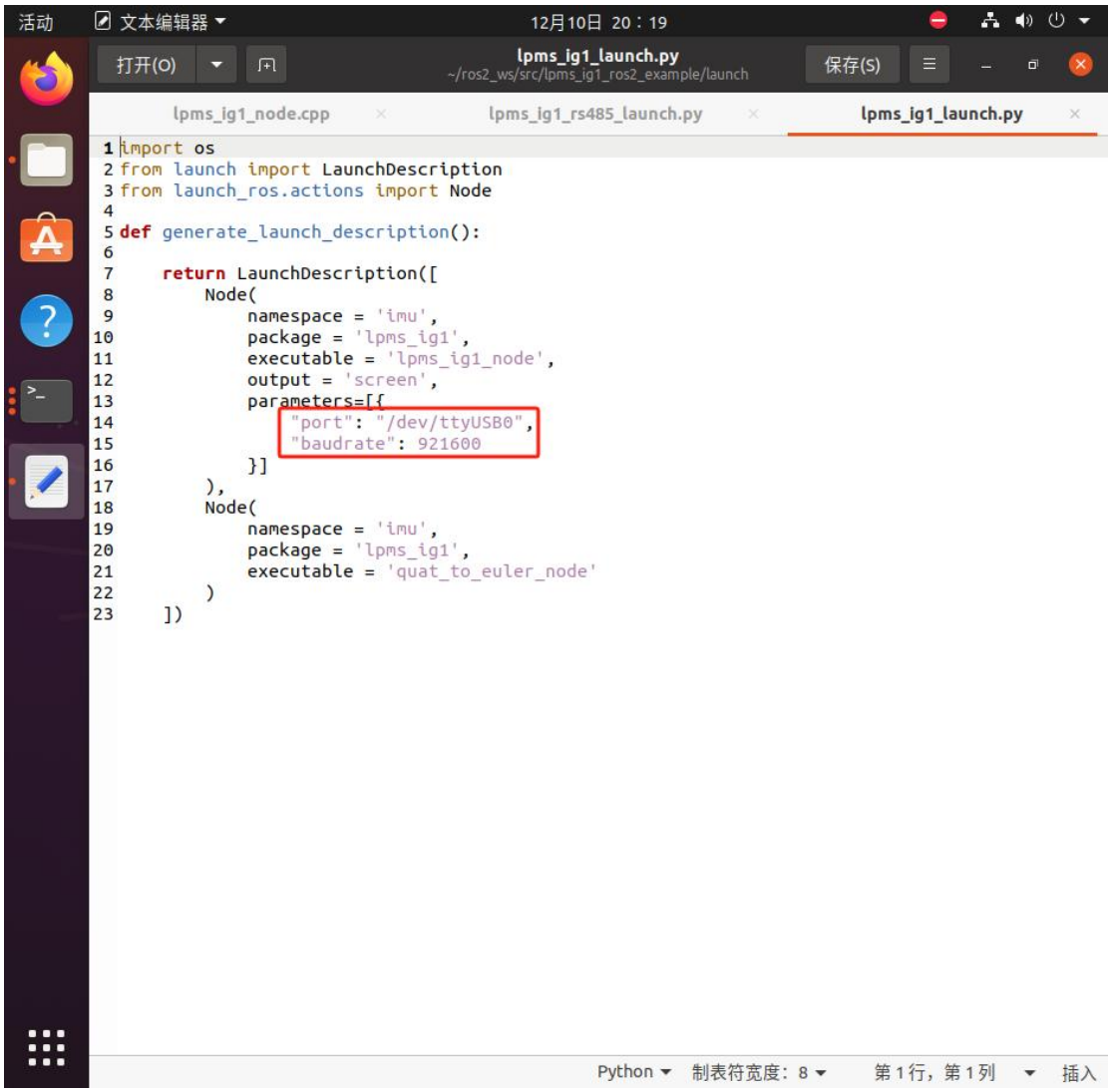
NAV3.系列（注意launch文件中的代码端口是以/dev/ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
ros2 launch lpms_ig1 lpms_nav3_launch.py
```

U.系列/CURS.系列/SI.系列/AL.系列（注意launch文件中的代码端口是以/dev/ttyUSB0为例子请根据实际情况修改，波特率请根据您的传感器实际设置更改参数）：

```
ros2 launch lpms_ig1 lpms_si1_launch.py
```

注意：运行前先查看对应的 launch 文件的端口和波特率是否正确



4.3.2 示例图片

```
lpms@lpms-virtual-machine:~/ros2_ws$ ros2 launch lpms_ig1 lpms_ig1_launch.py
[INFO] [launch]: All log files can be found below /home/lpms/.ros/log/2024-12-10-20-21-07-222729-lpms-virtual-machine-4174
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [lpms_ig1_node-1]: process started with pid [4178]
[INFO] [quat_to_euler_node-2]: process started with pid [4180]
[lpms_ig1_node-1] [INFO] [1733890867.299395851] [imu.lpms_ig1_node]: Settings
[lpms_ig1_node-1] [INFO] [1733890867.299458953] [imu.lpms_ig1_node]: Port: /dev/ttyUSB0
[lpms_ig1_node-1] [INFO] [1733890867.299464816] [imu.lpms_ig1_node]: Baudrate: 921600
[lpms_ig1_node-1] [INFO] [1733890867.299468379] [imu.lpms_ig1_node]: Auto reconnect: Enabled
[quat_to_euler_node-2] [INFO] [1733890867.305961982] [imu.quat_to_euler_node]: waiting for imu data
[lpms_ig1_node-1] [INFO] [1733890867.503433793] [imu.lpms_ig1_node]: Waiting for sensor to connect 1
[lpms_ig1_node-1] [INFO] [1733890868.504132924] [imu.lpms_ig1_node]: Sensor connected
```

4.4.已发布的主题(ros2 topic list 可查看)

4.4.1 示例图片

```
^Clpms@lpms-virtual-machine:~$ ros2 topic list
/imu/data
/imu/is_autocalibration_active
/imu/mag
/imu/rpy_angles
/parameter_events
/rosout
```

4.5 服务（ros2 service list 可查看）

4.5.1 示例图片

```
lpms@lpms-virtual-machine:~$ ros2 service list
/imu/calibrate_gyroscope
/imu/enable_auto_reconnect
/imu/enable_gyro_autocalibration
/imu/get_imu_data
/imu/lpms_ig1_node/describe_parameters
/imu/lpms_ig1_node/get_parameter_types
/imu/lpms_ig1_node/get_parameters
/imu/lpms_ig1_node/list_parameters
/imu/lpms_ig1_node/set_parameters
/imu/lpms_ig1_node/set_parameters_atomically
/imu/quat_to_euler_node/describe_parameters
/imu/quat_to_euler_node/get_parameter_types
/imu/quat_to_euler_node/get_parameters
/imu/quat_to_euler_node/list_parameters
/imu/quat_to_euler_node/set_parameters
/imu/quat_to_euler_node/set_parameters_atomically
/imu/reset_heading
/imu/set_command_mode
/imu/set_streaming_mode
```

4.5.2 常用服务节点解释

航向角归零

```
$ rosservice call /imu/reset_heading
```

陀螺仪手动校准

```
$ rosservice call /imu/calibrate_gyroscope
```

启开陀螺仪自动校准

```
$ rosservice call /imu/enable_gyro_autocalibration true
```

屏蔽陀螺仪自动校准

```
$ rosservice call /imu/enable_gyro_autocalibration false
```

用户可透过/imu/is_autocalibration_active Topic 来判断陀螺仪自动校准的状态

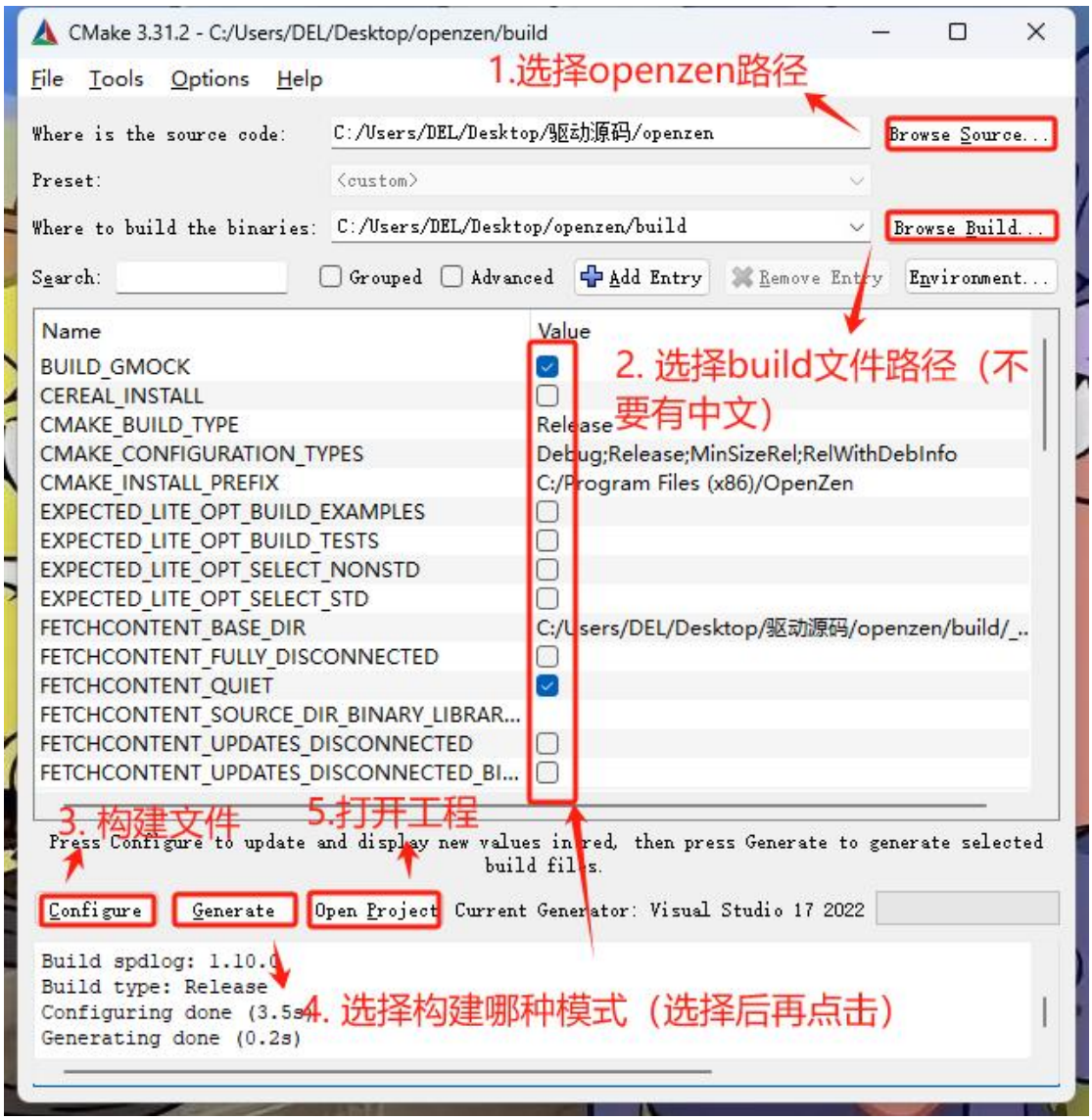
```
$ rostopic echo /imu/is_autocalibration_active
```

二、Openzen

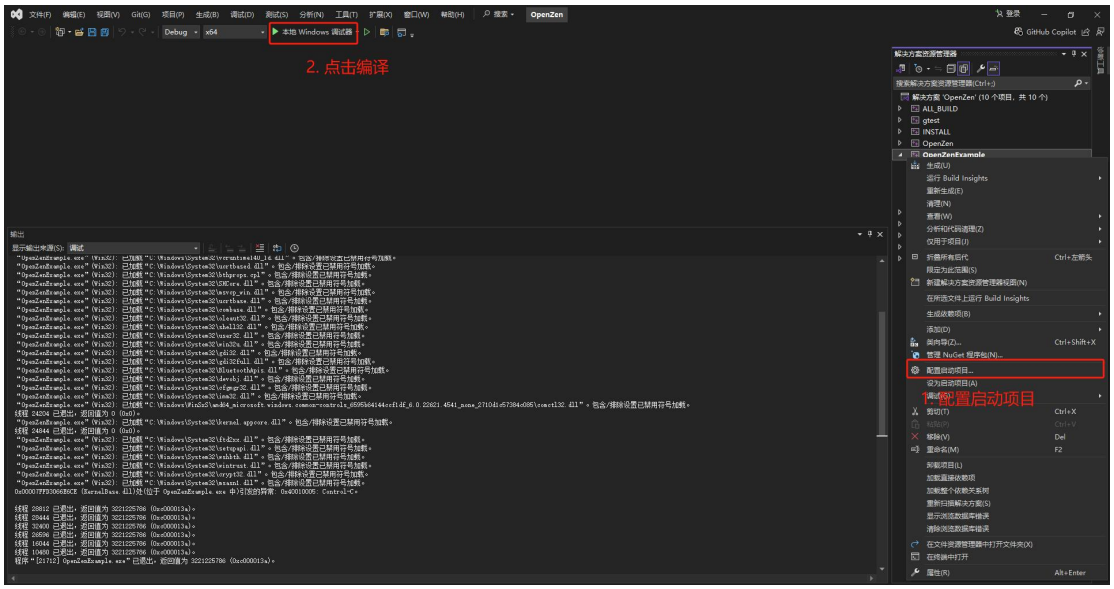
驱动开源网站：[lpresearch / openzen — Bitbucket](#) 使用时需自行 git
注意：需要在 windows 上下载 CMake 和 visualstudio 2022（其他版本也可以）

1. Windows 有线连接

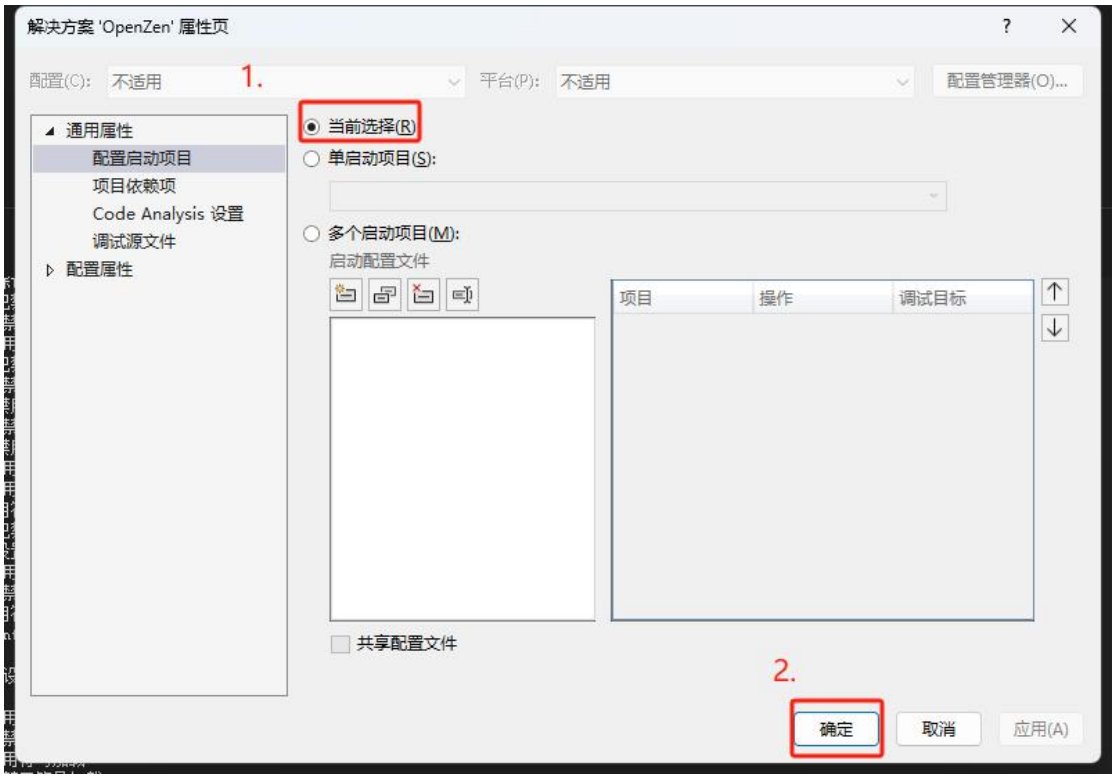
1.1 已经在网站上 git 下来之后，在 CMake 上按照图片步骤进行编译

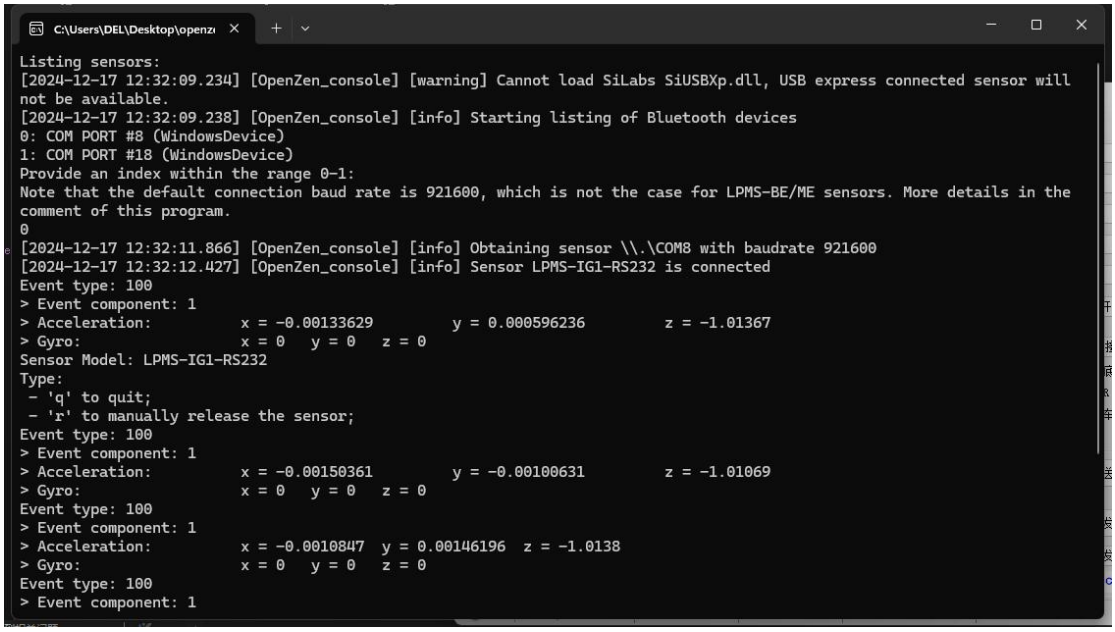


1.2 在打开工程之后按照图片中的步骤进行配置启动项目和编译

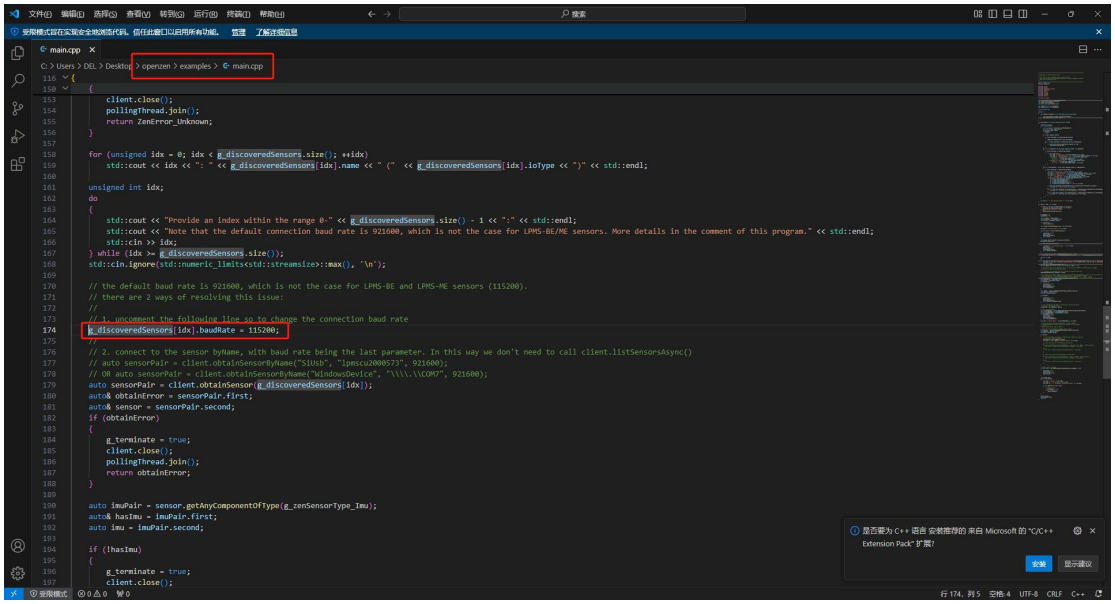


配置时会出现弹窗选择按下方图片中步骤即可





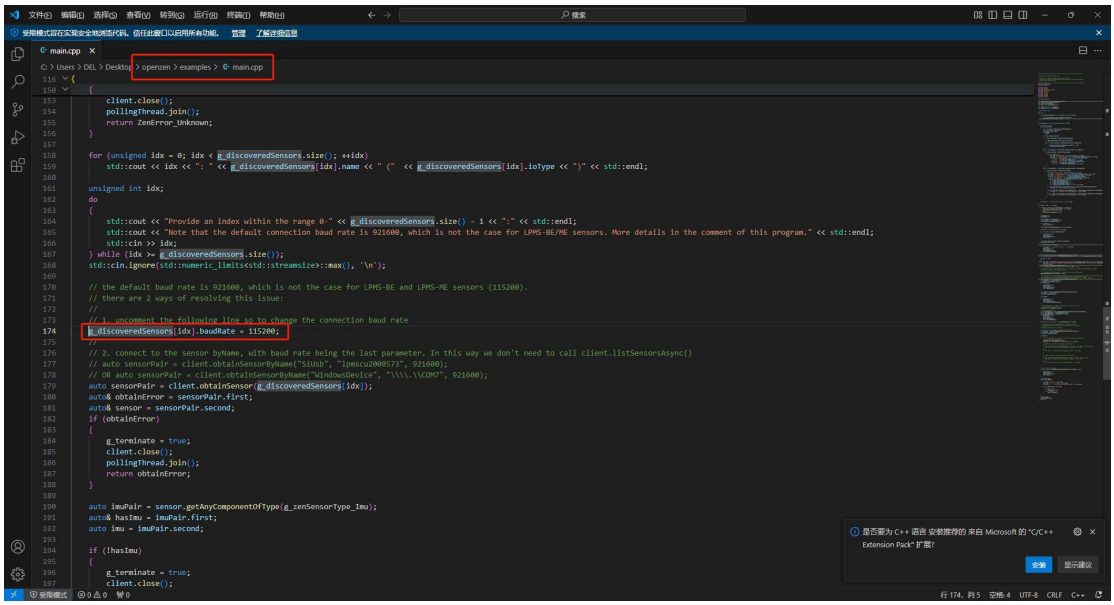
注意：默认的波特率是 921600 如果您的传感器是其他的波特率可以取消下方图中的注释并修改为对应的波特率



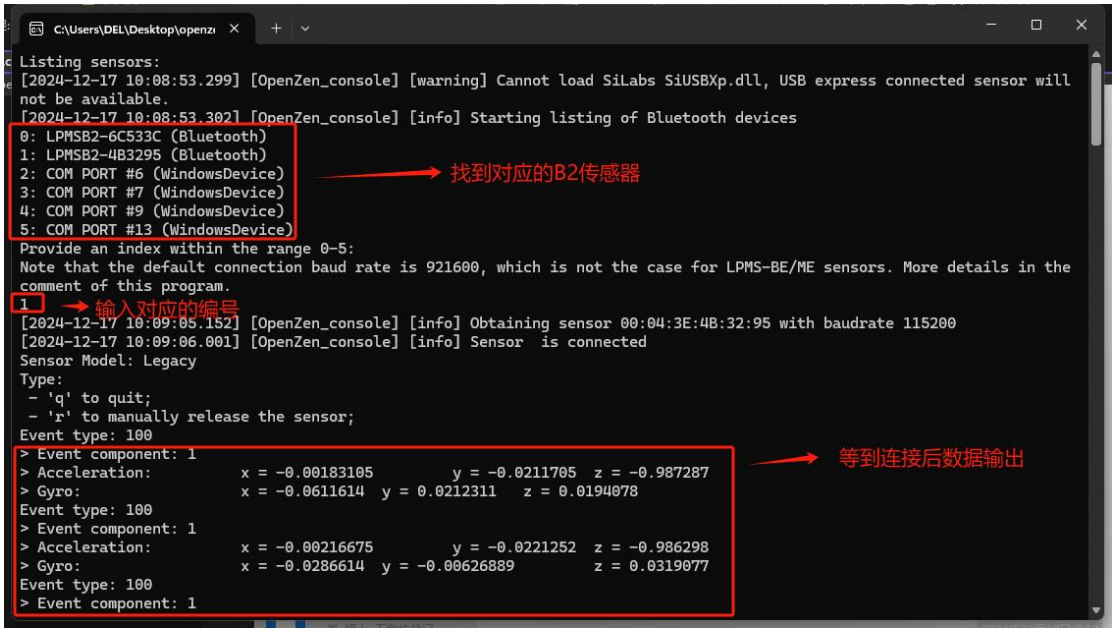
2. windows 蓝牙无线连接（B2）

2.1 打开电脑蓝牙

2.2 将图片中文件里面的语句取消注释



2.3 按照上面 1 的步骤编译完后再运行，按照图片中操作步骤进行即可



3. Linux 有线连接

2.1 步骤

Linux

1. 安装gcc7（需要c++ 17支持）或更新：‘sudo apt-get Install gcc-7’
2. 安装CMake（需要3.11或更高版本）
3. 克隆OpenZen存储库：‘git Clone——recurse-submodules https://bitbucket.org/lpresearch/openzen.git’
4. （适用于Linux）根据您的用例安装其他库。更多的细节(https://lpresearch.bitbucket.io/openzen/latest/setup.html linux)。
5. 创建一个构建文件夹并运行cmake：

```
cd openzen
mkdir build && cd build
cmake ..
```

6. 编译并运行*OpenZenExample*：

```
make -j4
examples/OpenZenExample
```

2.2 示例图片

```
melodic@melodic-virtual-machine:~/openzen/build$ cmake ..
CMake Deprecation Warning at external/expected-lite/CMakeLists.txt:8 (cmake_minimum_required):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value.  Or, use the <min>...<max> syntax
  to tell CMake that the project requires at least <min> but has been updated
  to work with policies introduced by <max> or earlier.


-- Project 'expected_lite', package 'expected-lite' version: '0.2.0'
CMake Deprecation Warning at external/gsl/CMakeLists.txt:1 (cmake_minimum_required):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value.  Or, use the <min>...<max> syntax
  to tell CMake that the project requires at least <min> but has been updated
  to work with policies introduced by <max> or earlier.
```

```
melodic@melodic-virtual-machine:~/openzen/build$ make -j4
[  2%] Built target gtest
[ 10%] Built target spdlog
[ 48%] Built target OpenZen
[ 51%] Built target OpenZenExampleCpp17
[ 52%] Building CXX object examples/CMakeFiles/OpenZenExampleCpp.dir/ExampleCpp.cpp.o
[ 53%] Building CXX object examples/CMakeFiles/OpenZenExample.dir/main.cpp.o
[ 97%] Built target OpenZenTests
[ 98%] Linking CXX executable OpenZenExampleCpp
[100%] Linking CXX executable OpenZenExample
[100%] Built target OpenZenExampleCpp
[100%] Built target OpenZenExample
```

```
melodic@melodic-virtual-machine:~/openzen/build$ examples/OpenZenExample
Listing sensors:
[2024-12-16 18:30:52.033] [OpenZen_console] [info] Found serial name IG1232000410034 for device /dev/ttyUSB0
[2024-12-16 18:30:52.033] [OpenZen_console] [info] Starting listing of Bluetooth devices
0: IG1232000410034 (LinuxDevice)
1: LPMSB2-4B3295 (Bluetooth)
2: LPMSB2-6C533C (Bluetooth)
Provide an index within the range 0-2:
Note that the default connection baud rate is 921600, which is not the case for LPMS-BE/ME sensors. More details in the comment of this program.
0
[2024-12-16 18:30:59.233] [OpenZen_console] [info] Obtaining sensor /dev/ttyUSB0 with baudrate 921600
[2024-12-16 18:30:59.233] [OpenZen_console] [info] Found serial name IG1232000410034 for device /dev/ttyUSB0
[2024-12-16 18:30:59.233] [OpenZen_console] [info] Opening file /dev/ttyUSB0 for sensor communication
[2024-12-16 18:30:59.488] [OpenZen_console] [info] Sensor LPMS-IG1-RS232 is connected
Sensor Model: LPMS-IG1-RS232
Type:
 - 'q' to quit;
 - 'r' to manually release the sensor;
Event type: 100
> Event component: 1
> Acceleration:          x = -0.00378539         y = -0.00100738         z = -1.01507
> Gyro:                  x = 0   y = 0   z = 0
Event type: 100
> Event component: 1
> Acceleration:          x = -0.00110272         y = -0.00126201         z = -1.01342
> Gyro:                  x = 0   y = 0   z = 0
Event type: 100
> Event component: 1
> Acceleration:          x = -0.000478762        y = 0.00096527  z = -1.01329
> Gyro:                  x = 0   y = 0   z = 0
```

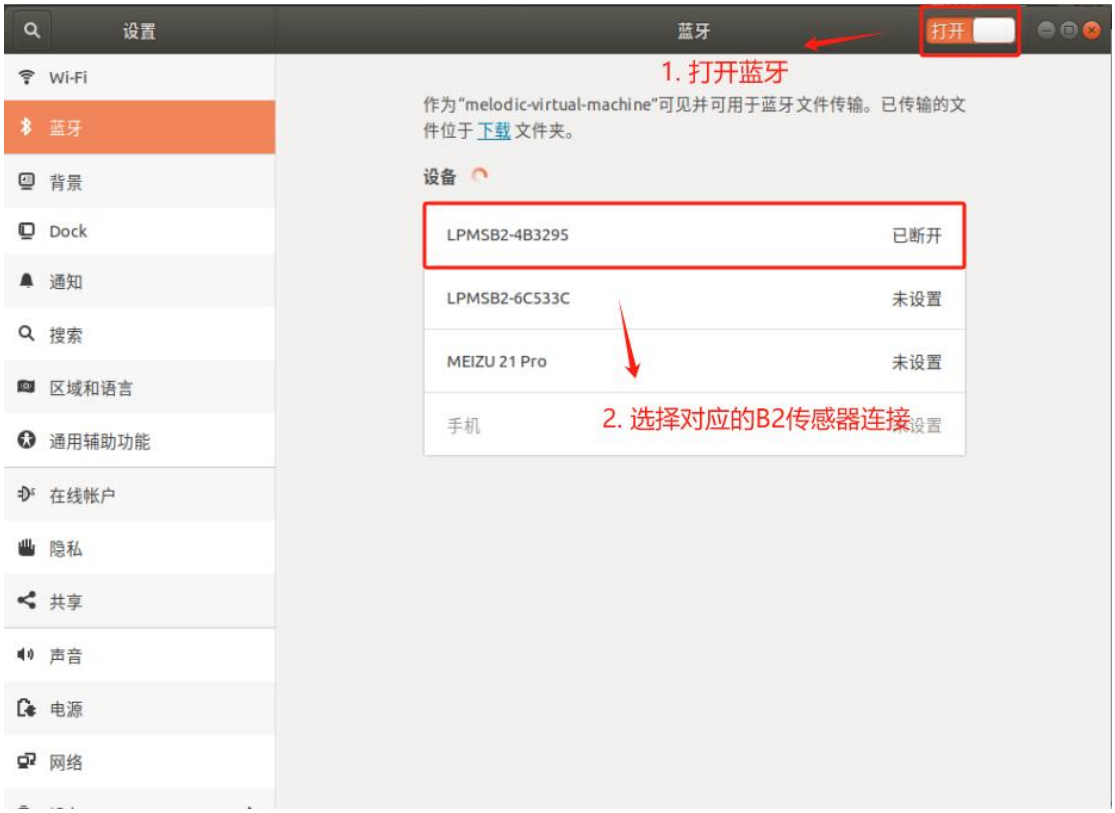
1.找到有线传感器对应的号码

2.输出对应号码

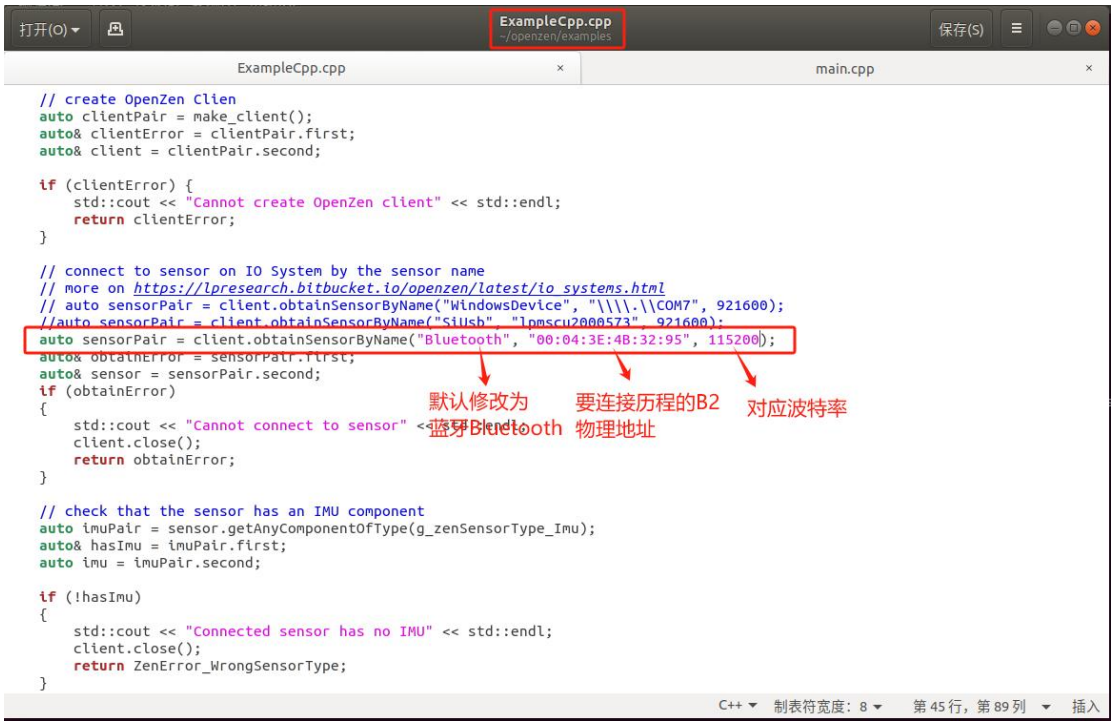
3.等待数据输出

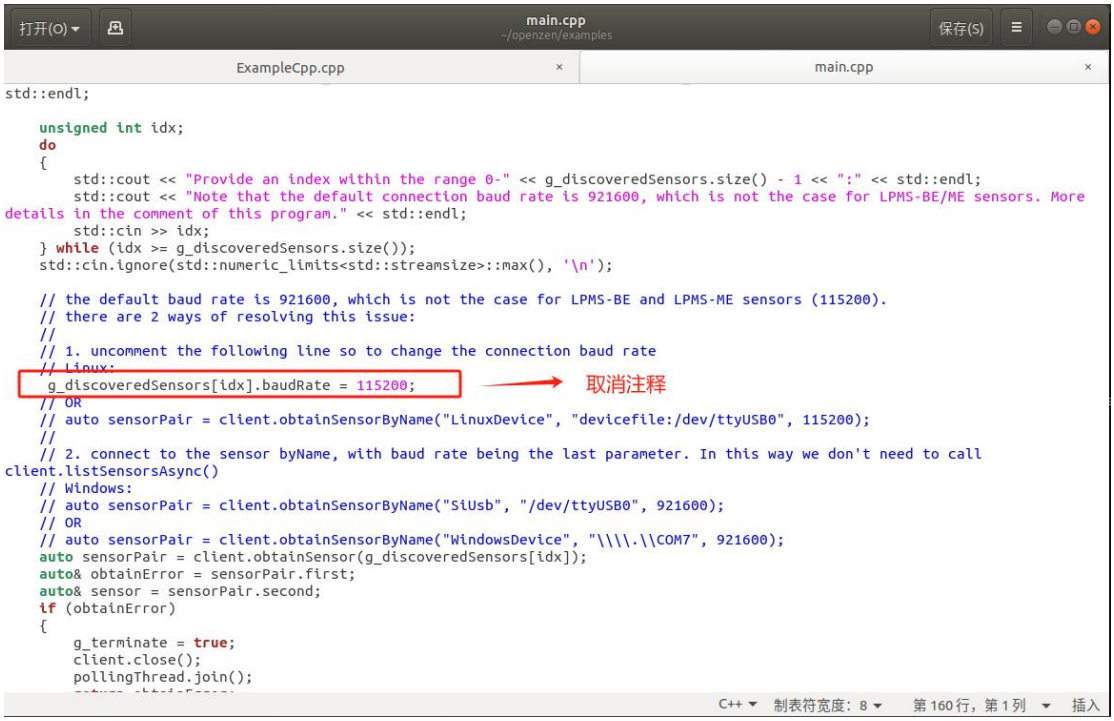
4. Linux 蓝牙无线连接（B2）

3.1 在 ubuntu 上插上蓝牙适配器并打开搜索到对应的蓝牙 B2



3.2 在 OPENZEN 上修改内容（可以通过 Ubuntu 的蓝牙查看 B2 的物理地址）





3.3 按照官网上面的步骤进行编译

Linux

1. Install gcc7 (requires C++17 support) or newer: `sudo apt-get install gcc-7`
2. Install CMake (requires version 3.11 or newer)
3. Clone the OpenZen repository: `git clone --recurse-submodules https://bitbucket.org/lpresearch/openzen.git`
4. (for Linux) Install additional libraries depending on your use case. More details.
5. Create a build folder and run cmake:

```
cd openzen
mkdir build && cd build
cmake ..
```

6. Compile and run the *OpenZenExample*:

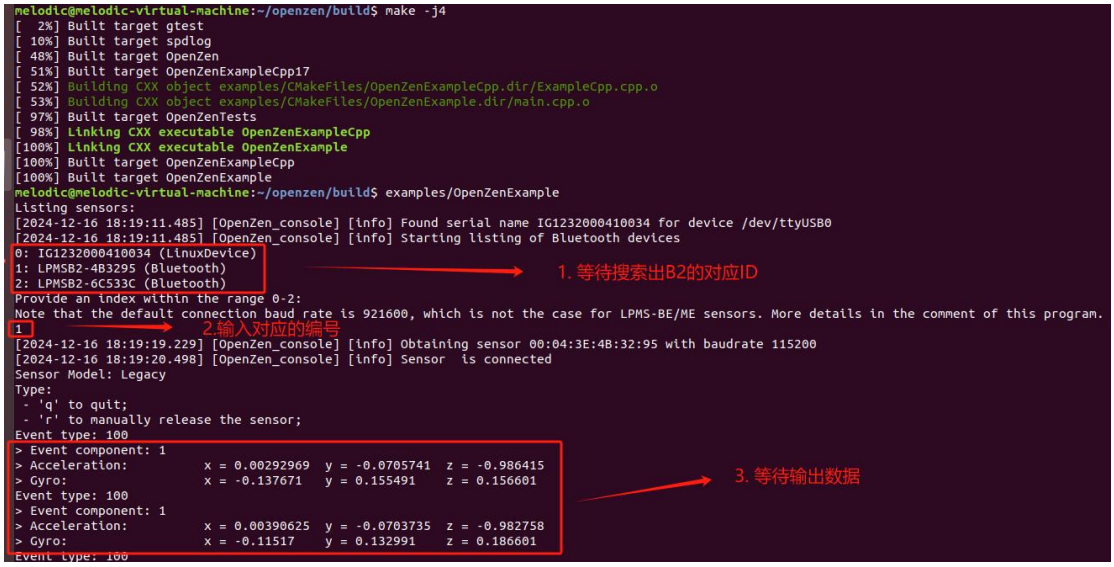
```
make -j4
examples/OpenZenExample
```

```
melodic@melodic-virtual-machine:~/openzen/build$ cmake ..
CMake Deprecation Warning at external/expected-lite/CMakeLists.txt:8 (cmake_minimum_required):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.

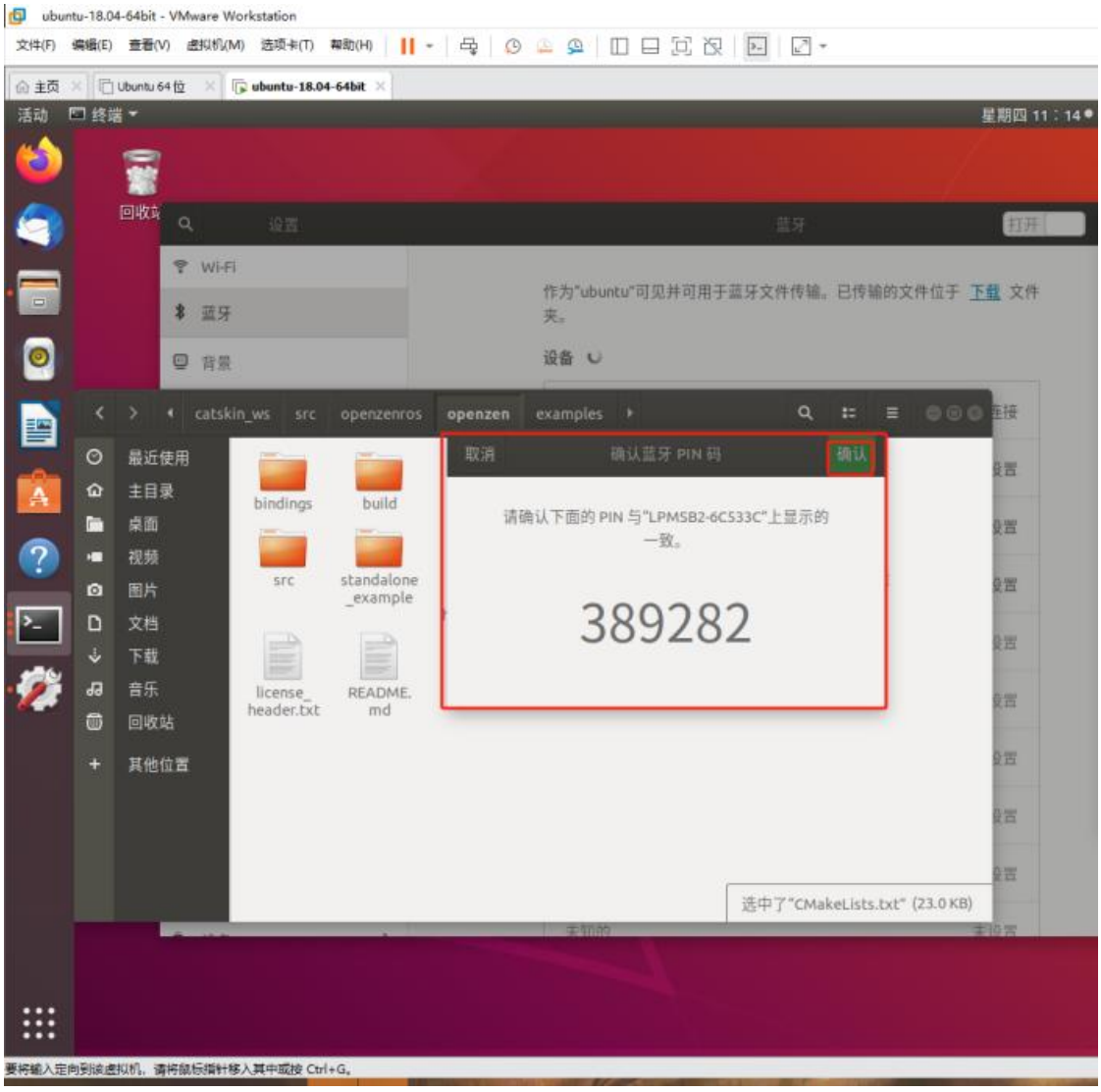
  Update the VERSION argument <min> value.  Or, use the <min>...<max> syntax
  to tell CMake that the project requires at least <min> but has been updated
  to work with policies introduced by <max> or earlier.


-- Project 'expected_lite', package 'expected-lite' version: '0.2.0'
CMake Deprecation Warning at external/gsl/CMakeLists.txt:1 (cmake_minimum_required):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value.  Or, use the <min>...<max> syntax
  to tell CMake that the project requires at least <min> but has been updated
  to work with policies introduced by <max> or earlier.
```



有时点击输入对应的编号后会弹出对话框确认蓝牙 PIN 码此时点击确认即可



三、Openzenros

驱动开源网站：[lpresearch / openzenros — Bitbucket]

1.编译 Library

1.1 步骤

要编译此软件，至少需要安装g++ 7.0版本和安装最新的cmake 版本。如果您的默认编译器是旧版本您可以使用发行版包管理器安装最新的GCC，并将GCC的名称提供给catkin_make命令

像这样:

```
catkin_make -DCMAKE_C_COMPILER=gcc-7 -DCMAKE_CXX_COMPILER=g++-7
```

要在ROS设置中编译此驱动程序，请遵循以下步骤:

```
mkdir -p catskin_ws/src
cd catskin_ws/src

git clone --recurse-submodules https://bitbucket.org/lpresearch/openzenros.git

# 编译ROS示例程序
source /opt/ros/melodic/setup.bash
cd ../../..
catkin_make
source ./devel/setup.bash

# 给端口权限(可以通过dmesg | grep tty之类的指令查看端口，示例中是USB0实际上可能是其他的)
sudo chmod 777 /dev/ttyUSB0
```

如果出现报错可以查看下方的报错排查

1.2 示例图片

```
melodic@melodic-virtual-machine:~$ mkdir -p catskin_ws/src
melodic@melodic-virtual-machine:~$ cd catskin_ws/src
melodic@melodic-virtual-machine:~/catskin_ws/src$
melodic@melodic-virtual-machine:~/catskin_ws/src$ git clone --recurse-submodules https://
bitbucket.org/lpresearch/openzenros.git
正克隆到 'openzenros'...
remote: Enumerating objects: 230, done.
remote: Counting objects: 100% (230/230), done.
remote: Compressing objects: 100% (229/229), done.
remote: Total 230 (delta 128), reused 0 (delta 0), pack-reused 0 (from 0)
接收对象中: 100% (230/230), 37.49 KiB | 336.00 KiB/s, 完成.
处理 delta 中: 100% (128/128), 完成.
子模组 'openzen' (https://bitbucket.org/lpresearch/openzen.git) 未对路径 'openzen' 注册
正克隆到 '/home/melodic/catskin_ws/src/openzenros/openzen'...
remote: Enumerating objects: 5727, done.
remote: Counting objects: 100% (5727/5727), done.
```

```
melodic@melodic-virtual-machine:~/catkin_ws/src$ source /opt/ros/melodic/setup.bash
melodic@melodic-virtual-machine:~/catkin_ws/src$ cd ..
melodic@melodic-virtual-machine:~/catkin_ws$ catkin_make
Base path: /home/melodic/catkin_ws
Source space: /home/melodic/catkin_ws/src
Build space: /home/melodic/catkin_ws/build
Devel space: /home/melodic/catkin_ws/devel
Install space: /home/melodic/catkin_ws/install
Creating symlink "/home/melodic/catkin_ws/src/CMakeLists.txt" pointing to "/opt/ros/melo
dic/share/catkin/cmake/toplevel.cmake"
####
#### Running command: "cmake /home/melodic/catkin_ws/src -DCATKIN_DEVEL_PREFIX=/home/mel
odic/catkin_ws/devel -DCMAKE_INSTALL_PREFIX=/home/melodic/catkin_ws/install -G Unix Mak
efiles" in "/home/melodic/catkin_ws/build"
####
CMake Deprecation Warning at CMakeLists.txt:4 (cmake_minimum_required):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.
```

```
[ 97%] Building CXX object openzenros/CMakeFiles/openzen_sensor_node.dir/src/OpenZenNode.
cpp.o
[100%] Linking CXX executable /home/melodic/catkin_ws/devel/lib/openzen_sensor/openzen_s
ensor_node
[100%] Built target openzen_sensor_node
melodic@melodic-virtual-machine:~/catkin_ws$ source ./devel/setup.bash
melodic@melodic-virtual-machine:~/catkin_ws$ sudo chmod 777 /dev/ttyUSB0
```

2. 运行 ROS 节点

2.1 步骤

打开另一个终端窗口，运行ROS内核：

```
source /opt/ros/melodic/setup.bash
roscore
```

然后可以在窗口中使用此命令运行OpenZen ROS驱动程序你曾经编译过的软件：

```
rosrun openzen_sensor openzen_sensor_node
```

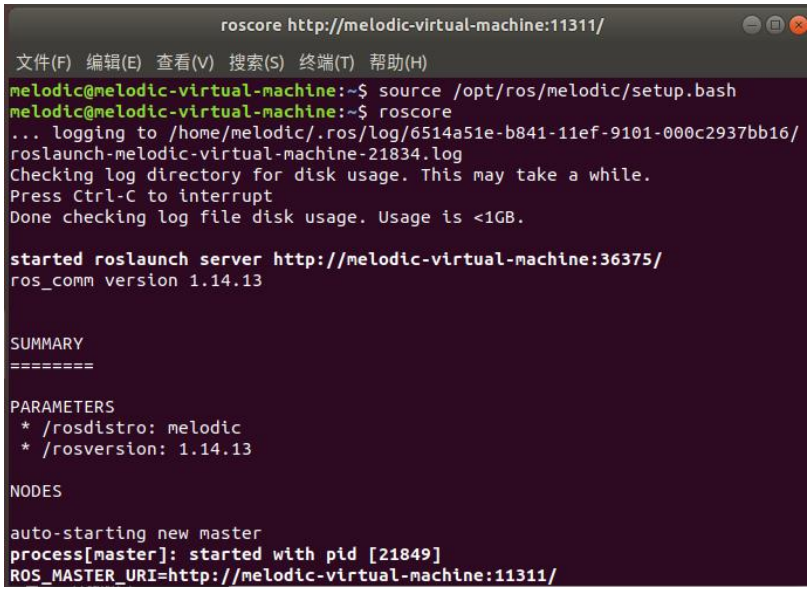
默认情况下，它将连接到第一个可用的传感器。如果你想连接到对于特定的传感器，可以使用传感器的串行名称作为参数，例如：
（注意示例中端口是以ttyUSB0为例子请根据实际情况修改）

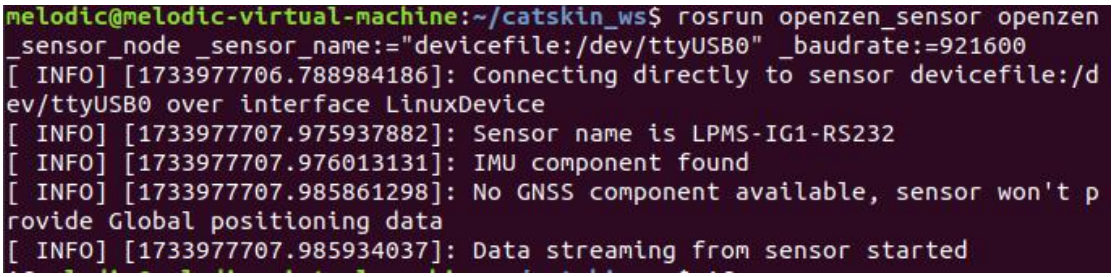
```
rosrun openzen_sensor openzen_sensor_node _sensor_name:="devicefile:/dev/ttyUSB0"
```

如果传感器配置为不同的波特率，请根据您的传感器设置适当更改参数
（各个系列IMU默认的波特率 IG1系列： 921600， IG1-RS485： 115200， BE系列： 115200，*NAV3系列*: 115200，U系列/CURS系列/SI系列/AL系列：921600；修改过的_baudrate:=填写修改后的波特率）：

```
rosrun openzen_sensor openzen_sensor_node _sensor_name:="devicefile:/dev/ttyUSB0"
_baudrate:=921600
```

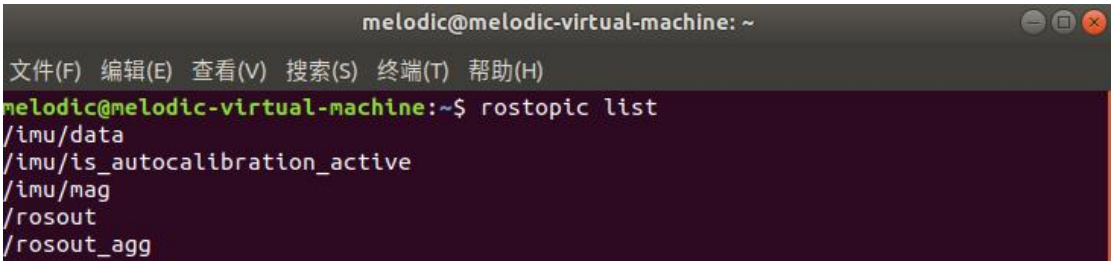
2.2 示例图片





3 已发布的主题(rostopic list 可查看)

3.4.1 示例图片



4 服务（rosservice list 可查看）

3.5.1 示例图片

```
melodic@melodic-virtual-machine:~$ rosservice list
/imu/calibrate_gyroscope
/imu/enable_gyro_autocalibration
/imu/reset_heading
/openzen_sensor_node/get_loggers
/openzen_sensor_node/set_logger_level
/rosout/get_loggers
/rosout/set_logger_level
melodic@melodic-virtual-machine:~$
```

3.5.2 常用服务节点解释

航向角归零

```
$ rosservice call /imu/reset_heading
```

陀螺仪手动校准

```
$ rosservice call /imu/calibrate_gyroscope
```

启开陀螺仪自动校准

```
$ rosservice call /imu/enable_gyro_autocalibration true
```

屏蔽陀螺仪自动校准

```
$ rosservice call /imu/enable_gyro_autocalibration false
```

用户可透过/imu/is_autocalibration_active Topic 来判断陀螺仪自动校准的状态

```
$ rostopic echo /imu/is_autocalibration_active
```

四、Openzenros2（foxy 版本为示例）

驱动开源网站：[lpresearch / openzenros2 — Bitbucket]

1.编译 Library

1.1 步骤

在此调用之后，您应该注销并使用此用户登录，以确保更改的权限生效。要允许访问通过 USB 连接的传感器，您需要确保运行 ROS 传感器节点的用户有权访问 /dev/ttyUSB 设备。您可以通过将用户添加到拨出组来执行此操作（可通过whoami来查看username）。

```
sudo adduser <username> dialout
```

要在ROS2设置中编译此驱动程序，请遵循以下步骤：

```
mkdir -p ros_ws/src
cd ros_ws/src

git clone --recurse-submodules https://bitbucket.org/lpresearch/openzenros2.git

# 编译ROS2示例程序
source /opt/ros/foxy/setup.bash
cd ..
colcon build
source ./install/setup.bash

# 给端口权限(可以通过dmesg | grep tty之类的指令查看端口，示例中是USB0实际上可能是其他的)
sudo chmod 777 /dev/ttyUSB0
```

如果出现报错可以查看下方的报错排查

1.2 示例图片

```
lpms@lpms-virtual-machine:~$ whoami
lpms
lpms@lpms-virtual-machine:~$ sudo adduser lpms dialout
[sudo] lpms 的密码：
正在添加用户"lpms"到"dialout"组...
正在将用户“lpms”加入到“dialout”组中
完成。
lpms@lpms-virtual-machine:~$ mkdir -p ros_ws/src
lpms@lpms-virtual-machine:~$ cd ros_ws/src
lpms@lpms-virtual-machine:~/ros_ws/src$ git clone --recurse-submodules https://bitbuc
ket.org/lpresearch/openzenros2.git
正克隆到 'openzenros2'...
remote: Enumerating objects: 73, done.
remote: Counting objects: 100% (73/73), done.
remote: Compressing objects: 100% (71/71), done.
remote: Total 73 (delta 31), reused 0 (delta 0), pack-reused 0 (from 0)
展开对象中: 100% (73/73), 17.81 KiB | 1.37 MiB/s, 完成.
子模组 'openzen' (https://bitbucket.org/lpresearch/openzen.git) 已对路径 'openzen' 注
册
正克隆到 '/home/lpms/ros_ws/src/openzenros2/openzen'...
```

```
子模组路径 'openzen/external/googletest': 检出 '62df7bdbc10887e094661e07ec2595b792037
6fd'
子模组路径 'openzen/external/gsl': 检出 '82ba6c77ceb70cf92add22ca9ecaf0fc7cac99fa'
子模组路径 'openzen/external/libzmq': 检出 '04f5bbedee58c538934374dc45182d8fc5926fa3'
子模组路径 'openzen/external/pybind11': 检出 '3e9dfa2866941655c56877882565e7577de6fc7
b'
子模组路径 'openzen/external/spdlog': 检出 '27cb4c76708608465c413f6d0e6b8d99a4d84302'
lpms@lpms-virtual-machine:~/ros_ws/src$ source /opt/ros/foxy/setup.bash
lpms@lpms-virtual-machine:~/ros_ws/src$ cd ...
bash: cd: ...: 没有那个文件或目录
lpms@lpms-virtual-machine:~/ros_ws/src$ cd ..
lpms@lpms-virtual-machine:~/ros_ws$ colcon build
Starting >>> openzen_driver
[Processing: openzen_driver]
--- stderr: openzen_driver
Please install libbluetooth-dev for bluetooth support
Binary libraries will not be downloaded
---
Finished <<< openzen_driver [52.7s]

Summary: 1 package finished [53.5s]
  1 package had stderr output: openzen_driver
lpms@lpms-virtual-machine:~/ros_ws$ sudo chmod 777 /dev/ttyUSB0
lpms@lpms-virtual-machine:~/ros_ws$
```

2. 运行 ROS2 节点

2.1 步骤

您现在可以在窗口中使用此命令运行OpenZen ROS2驱动程序你曾经编译过的软件：

```
ros2 run openzen_driver openzen_node --ros-args --remap __ns:=/openzen
```

默认情况下，它将连接到第一个可用的传感器。如果你想连接到对于特定的传感器，可以使用传感器的串行名称作为参数，例如：
（注意示例中端口是以ttyUSB0为例子请根据实际情况修改）

```
ros2 run openzen_driver openzen_node --ros-args --remap __ns:=/openzen -p
sensor_name:="devicefile:/dev/ttyUSB0"
```

如果传感器配置为不同的波特率，请根据您的传感器设置适当更改参数
（各个系列IMU默认的波特率 IG1系列：921600，IG1-RS485：115200，BE系列：115200，*NAV3系列*: 115200，U系列/CURS系列/SI系列/AL系列：921600；修改过的_baudrate:=填写修改后的波特率）：

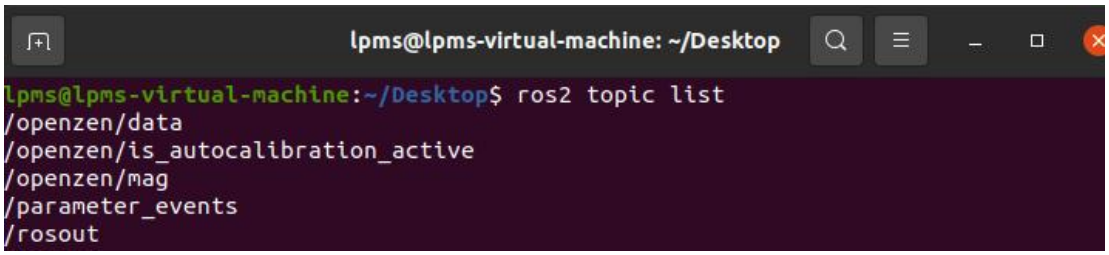
```
ros2 run openzen_driver openzen_node --ros-args --remap __ns:=/openzen -p
sensor_name:="devicefile:/dev/ttyUSB0" -p baudrate:=115200
```

2.2 示例图片

```
lpms@lpms-virtual-machine:~/ros_ws$ source ./install/setup.bash
lpms@lpms-virtual-machine:~/ros_ws$ ros2 run openzen_driver openzen_node -
-ros-args --remap __ns:=/openzen -p sensor_name:="devicefile:/dev/ttyUSB0"
 -p baudrate:=921600
[INFO] [1733976299.588048449] [openzen.openzen_node]: Connecting directly
to sensor devicefile:/dev/ttyUSB0 over interface LinuxDevice
[INFO] [1733976301.805007826] [openzen.openzen_node]: IMU component found
[INFO] [1733976301.813245071] [openzen.openzen_node]: No GNSS component av
ailable, sensor won't provide Global positioning data
[INFO] [1733976301.813400560] [openzen.openzen_node]: Data streaming from
sensor started
```

3 已发布的主题(ros2 topic list 可查看)

3.4.1 示例图片



4 服务（ros2 service list 可查看）

3.5.1 示例图片



3.5.2 常用服务节点解释

/openzen/calibrate_gyroscope 陀螺仪手动标定

/openzen/enable_gyro_autocalibration 开启陀螺仪自动标定运行时需要后面加 true，关闭需要在后面加 false

/openzen/reset_heading 航向角归零

5 Humble 版本编译遇到问题有两个解决办法

5.1.1 修改分支（优先使用这个方法）

```
C:\Users\DEL\Desktop\驱动源码\openzenros2>git branch
* master

C:\Users\DEL\Desktop\驱动源码\openzenros2>git checkout hotfix/build_on_ros2_humble
M       openzen
branch 'hotfix/build_on_ros2_humble' set up to track 'origin/hotfix/build_on_ros2_humble'.
Switched to a new branch 'hotfix/build_on_ros2_humble'

C:\Users\DEL\Desktop\驱动源码\openzenros2>git branch
* hotfix/build_on_ros2_humble
  master
```

5.1.2 重新编译

```
lpms@lpms-virtual-machine:~/new ros_ws$ colcon build
Starting >>> openzen_driver
[Processing: openzen_driver]
--- stderr: openzen_driver
Please install libbluetooth-dev for bluetooth support
Binary libraries will not be downloaded
---
Finished <<< openzen_driver [52.3s]

Summary: 1 package finished [52.5s]
  1 package had stderr output: openzen_driver
```

5.2.1 按照图片位置对相对于的代码进行注释




```
170     endif()
171 else()
172     add_library(spdlog STATIC ${SPDLOG_SRCS} ${SPDLOG_ALL_HEADERS})
173 endif()
174
175 #add_library(spdlog::spdlog ALIAS spdlog)
176
177 set(SPDLOG_INCLUDES_LEVEL "")
178 if(SPDLOG_SYSTEM_INCLUDES)
179     set(SPDLOG_INCLUDES_LEVEL "SYSTEM")
180 endif()
181
182 target_compile_definitions(spdlog PUBLIC SPDLOG_COMPILED_LIB)
183 target_include_directories(spdlog ${SPDLOG_INCLUDES_LEVEL} PUBLIC "$<BUILD_INTERFACE:$
    {CMAKE_CURRENT_LIST_DIR}/include>"
184                                                                 "$<INSTALL_INTERFACE:
    ${CMAKE_INSTALL_INCLUDEDIR}>")
185 target_link_libraries(spdlog PUBLIC Threads::Threads)
186 spdlog_enable_warnings(spdlog)
187
188 set_target_properties(spdlog PROPERTIES VERSION ${SPDLOG_VERSION} SOVERSION
189                                                                   $
    {SPDLOG_VERSION_MAJOR}.${SPDLOG_VERSION_MINOR})
190 set_target_properties(spdlog PROPERTIES DEBUG_POSTFIX d)
191
192 if(COMMAND target_precompile_headers AND SPDLOG_ENABLE_PCH)
193     configure_file(${CMAKE_CURRENT_SOURCE_DIR}/cmake/pch.h.in ${PROJECT_BINARY_DIR}/
    spdlog_pch.h @ONLY)
194     target_precompile_headers(spdlog PRIVATE ${PROJECT_BINARY_DIR}/spdlog_pch.h)
195 endif()
196
197 #
    ------------------------------------------------------------------------------------------
198 # Header only version
199 #
    ------------------------------------------------------------------------------------------
200 add_library(spdlog_header_only INTERFACE)
201 #add_library(spdlog::spdlog_header_only ALIAS spdlog_header_only)
202
203 target_include_directories(
204     spdlog_header_only ${SPDLOG_INCLUDES_LEVEL} INTERFACE "$<BUILD_INTERFACE:$
    {CMAKE_CURRENT_LIST_DIR}/include>"
205                                                           "$<INSTALL_INTERFACE:$
    {CMAKE_INSTALL_INCLUDEDIR}>")
206 target_link_libraries(spdlog_header_only INTERFACE Threads::Threads)
```




```
84         spdlog::set_default_logger(console_logger);
85     }
86
87     if (logLevel == ZenLogLevel::ZenLogLevel_Off) {
88         spdlog::set_level(spdlog::level::off);
89     }
90     else if (logLevel == ZenLogLevel::ZenLogLevel_Debug) {
91         spdlog::set_level(spdlog::level::debug);
92     }
93     else if (logLevel == ZenLogLevel::ZenLogLevel_Info) {
94         spdlog::set_level(spdlog::level::info);
95     }
96     else if (logLevel == ZenLogLevel::ZenLogLevel_Warning) {
97         spdlog::set_level(spdlog::level::warn);
98     }
99     else if (logLevel == ZenLogLevel::ZenLogLevel_Error) {
100        spdlog::set_level(spdlog::level::err);
101    } else {
102        //spdlog::error("Log Level {} not suppored", fmt::underlying(logLevel));
103        return ZenError_InvalidArgument;
104    }
105
106    return ZenError_None;
107 }
108
109 ZEN_API ZenError ZenListSensorsAsync(ZenClientHandle_t handle)
110 {
111     if (auto client = getClient(handle))
112     {
113         client->listSensorsAsync();
114         return ZenError_None;
115     }
116     else
117     {
118         return ZenError_InvalidClientHandle;
119     }
120 }
121
122 ZEN_API ZenSensorInitError ZenObtainSensor(ZenClientHandle_t clientHandle, const
    ZenSensorDesc* const desc, ZenSensorHandle_t* outSensorHandle)
123 {
124     if (desc == nullptr)
125         return ZenSensorInitError_IsNull;
126
127     if (outSensorHandle == nullptr)
```

Open ModbusCommunicator.cpp ~/ros_ws/src/openzenros2/openzen/src/communication Save

```
53          while (!data.empty())
54          {
55              while (m_parserBusy.test_and_set(std::memory_order_acquire)) { /*spin
    lock*/ }
56              auto guard = finally([this]() {
57                  m_parserBusy.clear(std::memory_order_release);
58              });
59
60              if (auto parserError = m_parser->parse(data); parserError !=
    modbus::FrameParseError_None)
61              {
62              /*
63                  spdlog::debug("Parsing of packet failed, can happen when OpenZen started
    to parse in the middle of a package. Error: {}",
64                      fmt::underlying(parserError));*/
65                  // drop first byte and look for new start character
66                  m_parser->reset();
67                  data = data.subspan(1);
68
69                  continue;
70              }
71
72              if (m_parser->finished())
73              {
74                  const auto& frame = m_parser->frame();
75
76                  spdlog::debug("Received and parsed message with address {} function {}
    and data size {}",
77                      frame.address, frame.function, frame.data.size());
78
79                  if (auto error = m_subscriber->processReceivedData(frame.address,
    frame.function, frame.data); error && error != ZenError_BufferTooSmall)
80                  {
81                      //spdlog::error("Failed to process message with address {} function
    {} data {}. Error: {}",
82                          //frame.address, frame.function, util::spanToString(frame.data),
    fmt::underlying(error));
83                  }
84                  m_parser->reset();
85              }
86          }
87
88          return ZenError_None;
89      }
90  }
```

C++ Tab Width: 8 Ln 1, Col 1 INS

5.2.2 重新编译

```
lpms@lpms-virtual-machine:~/new ros_ws$ colcon build
Starting >>> openzen_driver
[Processing: openzen_driver]
--- stderr: openzen_driver
Please install libbluetooth-dev for bluetooth support
Binary libraries will not be downloaded
---
Finished <<< openzen_driver [52.3s]

Summary: 1 package finished [52.5s]
  1 package had stderr output: openzen_driver
```

五、Lpmsnav2sopensourcelib

驱动开源网站：[lpresearch / lpmsnav2opensourcelib — Bitbucket]

1. 编译 Library

1.1 步骤

```
$ cd ~
$ git clone https://bitbucket.org/lpresearch/lpmsnav2opensourcelib
$ cd lpmsnav2opensourcelib
$ mkdir build
$ cd build
$ cmake ..
$ make
$ make package
$ sudo dpkg -i libLpmsNAV2_OpenSource-x.y.z-Linux.deb(X Y Z为make package下来的版本)
```

1.2 示例图片

```
melodic@melodic-virtual-machine:~$ git clone https://bitbucket.org/lpresear
ch/lpmsnav2opensourcelib
正克隆到 'lpmsnav2opensourcelib'...
remote: Enumerating objects: 77, done.
remote: Counting objects: 100% (77/77), done.
remote: Compressing objects: 100% (70/70), done.
remote: Total 77 (delta 39), reused 0 (delta 0), pack-reused 0 (from 0)
展开对象中: 100% (77/77), 完成.
melodic@melodic-virtual-machine:~$ cd lpmsnav2opensourcelib
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib$ mkdir build
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib$ cd build
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib/build$ cmake ..
CMake Deprecation Warning at CMakeLists.txt:32 (cmake_minimum_required):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value.  Or, use the <min>...<max> synta
x
  to tell CMake that the project requires at least <min> but has been updat
ed
  to work with policies introduced by <max> or earlier.


-- The C compiler identification is GNU 7.5.0
-- The CXX compiler identification is GNU 7.5.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done (0.3s)
-- Generating done (0.0s)
-- Build files have been written to: /home/melodic/lpmsnav2opensourcelib/bu
ild
```

```
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib/build$ make
[ 20%] Building CXX object CMakeFiles/LpmsNav2_OpenSourceLib.dir/MicroMeasu
re.o
[ 40%] Building CXX object CMakeFiles/LpmsNav2_OpenSourceLib.dir/SerialPort
Linux.o
[ 60%] Building CXX object CMakeFiles/LpmsNav2_OpenSourceLib.dir/LpmsNAV.o
[ 80%] Building CXX object CMakeFiles/LpmsNav2_OpenSourceLib.dir/util.o
[100%] Linking CXX shared library libLpmsNav2_OpenSourceLib.so
[100%] Built target LpmsNav2_OpenSourceLib
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib/build$ make package
[100%] Built target LpmsNav2_OpenSourceLib
Run CPack packaging tool...
CPack: Create package using DEB
CPack: Install projects
CPack: - Run preinstall target for: LpmsNav2_OpenSourceLib
CPack: - Install project: LpmsNav2_OpenSourceLib []
CPack: Create package
-- CPACK_DEBIAN_PACKAGE_DEPENDS not set, the package will have no dependenc
ies.
CPack: - package: /home/melodic/lpmsnav2opensourcelib/build/libLpmsNav2_Ope
nSource-2.0.1-Linux.deb generated.
```

```
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib/build$ ls
CMakeCache.txt       CPackSourceConfig.cmake
CMakeFiles           install_manifest.txt
cmake_install.cmake  libLpmsNav2_OpenSource-2.0.1-Linux.deb
CPackConfig.cmake    libLpmsNav2_OpenSourceLib.so
_CPack_Packages      Makefile
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib/build$ sudo dpkg -i
 libLpmsNav2_OpenSource-2.0.1-Linux.deb
正在选中未选择的软件包 liblpmsnav2_opensource。
(正在读取数据库 ... 系统当前共安装有 236497 个文件和目录。)
正准备解包 libLpmsNav2_OpenSource-2.0.1-Linux.deb  ...
正在解包 liblpmsnav2_opensource (2.0.1) ...
正在设置 liblpmsnav2_opensource (2.0.1) ...
```

2 编译 linux 示例程序

2.1 步骤

```
$ cd linux_example
$ mkdir build
$ cd build
$ cmake ..
$ make
$ sudo chmod 777 /dev/ttyUSB0(以实际的端口为主)
$ ./LpmsNAVSimpleExample </dev/ttyUSB0>
```

2.2 示例图片

```
melodic@melodic-virtual-machine:~$ cd lpmsnav2opensourcelib/
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib$ cd linux_example/
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib/linux_example$ mkdi
r build
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib/linux_example$ cd b
uild/
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib/linux_example/build
$ cmake ..
CMake Deprecation Warning at CMakeLists.txt:1 (cmake_minimum_required):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value.  Or, use the <min>...<max> synta
x
  to tell CMake that the project requires at least <min> but has been updat
ed
  to work with policies introduced by <max> or earlier.
```

```
-- Configuring done (0.2s)
-- Generating done (0.0s)
-- Build files have been written to: /home/melodic/lpmsnav2opensourcelib/li
nux_example/build
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib/linux_example/build
$ make
[ 50%] Building CXX object CMakeFiles/LpmsNavSimpleExample.dir/main.o
[100%] Linking CXX executable LpmsNavSimpleExample
[100%] Built target LpmsNavSimpleExample
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib/linux_example/build
$ sudo chmod 777 /dev/ttyUSB1
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib/linux_example/build
$ ls
CMakeCache.txt  cmake_install.cmake   Makefile
CMakeFiles      LpmsNavSimpleExample
```

```
melodic@melodic-virtual-machine:~/lpmsnav2opensourcelib/linux_example/build
$ ./LpmsNavSimpleExample /dev/ttyUSB1
Main Menu
==================
[i] Print sensor info
[p] Print sensor data
[c] Go to command mode
[s] Go to streaming mode
[h] Reset sensor heading
[b] Calibrate gyro static bias
[e] Enable gyro autocalibration
[d] Disable gyro autocalibration
[r] Reset factory settings
[q] quit
```

2.3 运行结果

```
p
[Main] Print sensor data
[Main] t:42.00 angle^C18.30 rate: 0.00 Hz: 100.014014
```

3.编译 ROS 示例程序

3.1 步骤

```
# Create ROS workspace.
    $ mkdir -p ~/catkin_ws_nav/src
    $ cd ~/catkin_ws_nav/src

# We suggest to create a symbolic link to ros_example folder instead of copying
    $ ln -s ~/lpmsnav2opensourcelib/ros_example lpms_nav2_ros_example

# Compiling ROS example programs
    $ cd ~/catkin_ws_nav
    $ catkin_make
    $ source ./devel/setup.bash
```

3.2 示例图片

```
melodic@melodic-virtual-machine:~$ mkdir -p ~/catkin_ws_nav/src
melodic@melodic-virtual-machine:~$ cd ~/catkin_ws_nav/src
melodic@melodic-virtual-machine:~/catkin_ws_nav/src$ ln -s ~/lpmsnav2openso
urcelib/ros_example lpms_nav2_ros_example
melodic@melodic-virtual-machine:~/catkin_ws_nav/src$ cd ~/catkin_ws_nav
melodic@melodic-virtual-machine:~/catkin_ws_nav$ catkin_make
Base path: /home/melodic/catkin_ws_nav
Source space: /home/melodic/catkin_ws_nav/src
Build space: /home/melodic/catkin_ws_nav/build
Devel space: /home/melodic/catkin_ws_nav/devel
Install space: /home/melodic/catkin_ws_nav/install
Creating symlink "/home/melodic/catkin_ws_nav/src/CMakeLists.txt" pointing
to "/opt/ros/melodic/share/catkin/cmake/toplevel.cmake"
####
#### Running command: "cmake /home/melodic/catkin_ws_nav/src -DCATKIN_DEVEL
_PREFIX=/home/melodic/catkin_ws_nav/devel -DCMAKE_INSTALL_PREFIX=/home/melo
dic/catkin_ws_nav/install -G Unix Makefiles" in "/home/melodic/catkin_ws_na
v/build"
####
CMake Deprecation Warning at CMakeLists.txt:4 (cmake_minimum_required):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value.  Or, use the <min>...<max> synta
x
  to tell CMake that the project requires at least <min> but has been updat
ed
```

```
[ 50%] Building CXX object lpms_nav2_ros_example/CMakeFiles/lpms_nav_node.d
ir/src/lpms_nav_node.cpp.o
[100%] Linking CXX executable /home/melodic/catkin_ws_nav/devel/lib/lpms_na
v/lpms_nav_node
[100%] Built target lpms_nav_node
melodic@melodic-virtual-machine:~/catkin_ws_nav$ source ./devel/setup.bash
melodic@melodic-virtual-machine:~/catkin_ws_nav$ sudo chmod 777 /dev/ttyUSB
0
[sudo] melodic 的密码:
```

4. 运行 ROS 节点

4.1 步骤

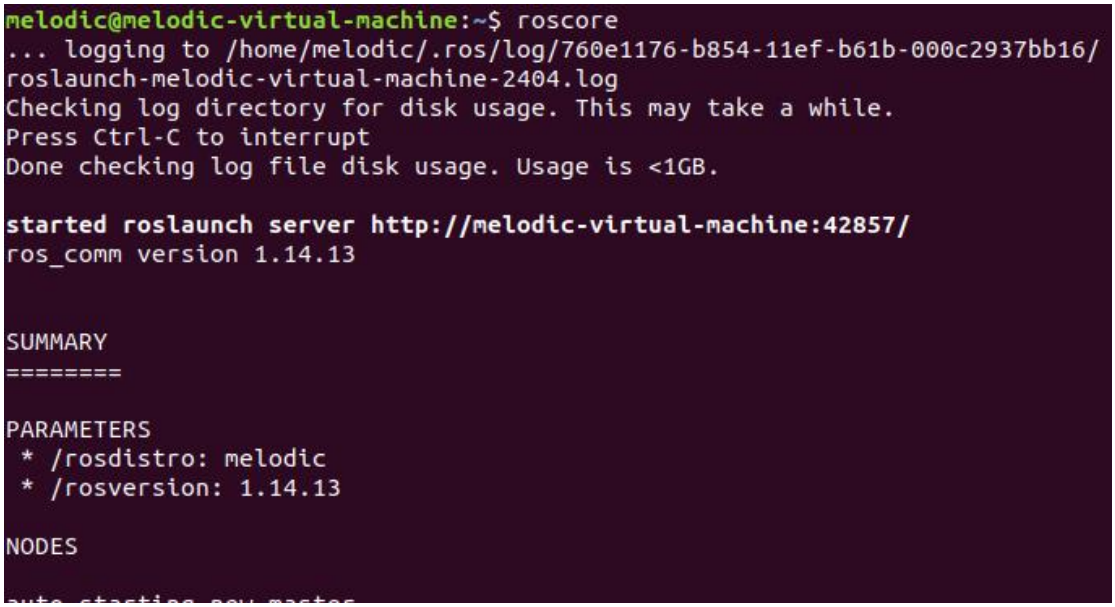
Open a new terminal window and run roscore

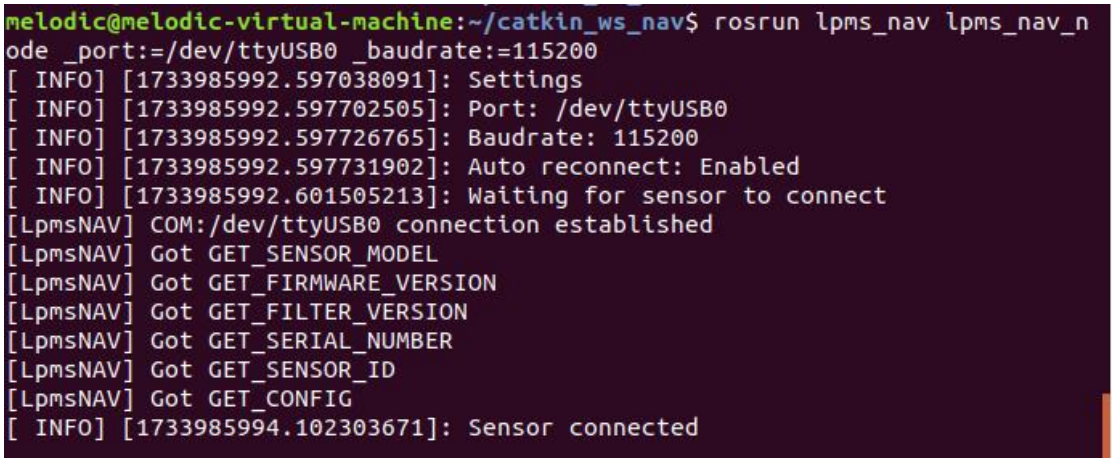
```
$ roscore
```

Run lpms_nav_node

```
$ rosrun lpms_nav lpms_nav_node _port:=/dev/ttyUSB0 _baudrate:=115200
```

4.2 示例图片





5.已发布的主题（rostopic list 可查看）

5.1 示例图片

```
melodic@melodic-virtual-machine:~$ rostopic list
/imu/data
/imu/is_autocalibration_active
/imu/rpy
/rosout
/rosout_agg
```

6.服务（rosservice list 可查看）

6.1 示例图片

```
melodic@melodic-virtual-machine:~$ rosservice list
/imu/calibrate_gyroscope
/imu/reset_heading
/imu/send_getSensorData
/imu/setAutoCalibrateStatus
/lpms_nav_node/get_loggers
/lpms_nav_node/set_logger_level
/rosout/get_loggers
/rosout/set_logger_level
```

6.2 常用服务节点解释

航向角归零

```
$ rosservice call /imu/reset_heading
```

陀螺仪手动校准

```
$ rosservice call /imu/calibrate_gyroscope
```

启开陀螺仪自动校准

```
$ rosservice call /imu/enable_gyro_autocalibration true
```

屏蔽陀螺仪自动校准

```
$ rosservice call /imu/enable_gyro_autocalibration false
```

用户可透过/imu/is_autocalibration_active Topic 来判断陀螺仪自动校准的状态

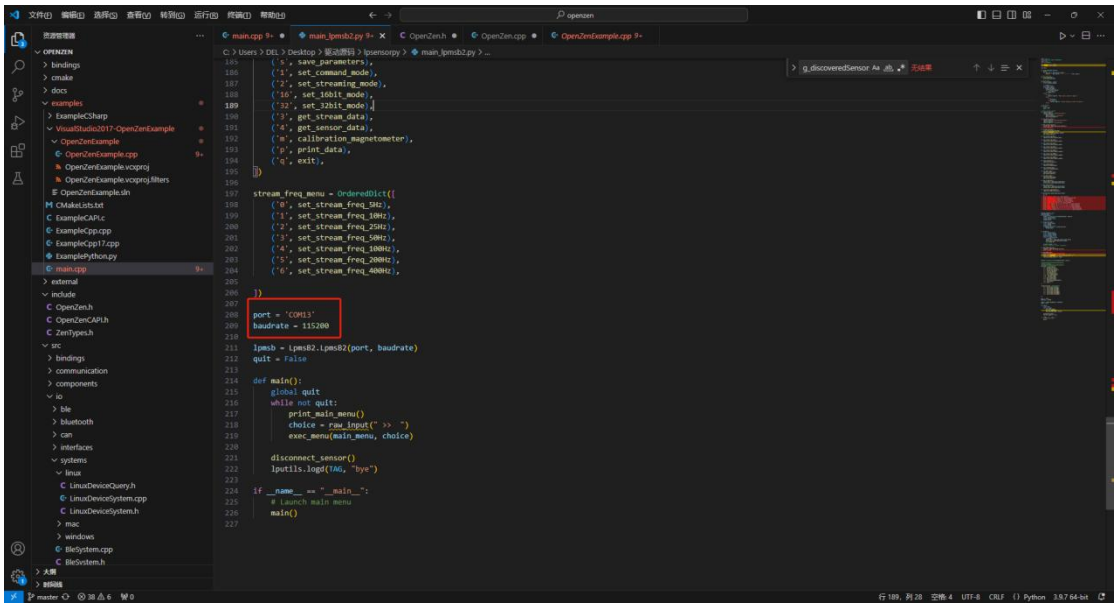
```
$ rostopic echo /imu/is_autocalibration_active
```

六、LpSensorPy

注意：由于代码需要 python2 用 python3 会报错，所以需要 python2 来运行
驱动开源网站：[lpresearch / lpsensorpy — Bitbucket]

windows 蓝牙无线连接（B2）

1. 打开文件修改对应端口波特率

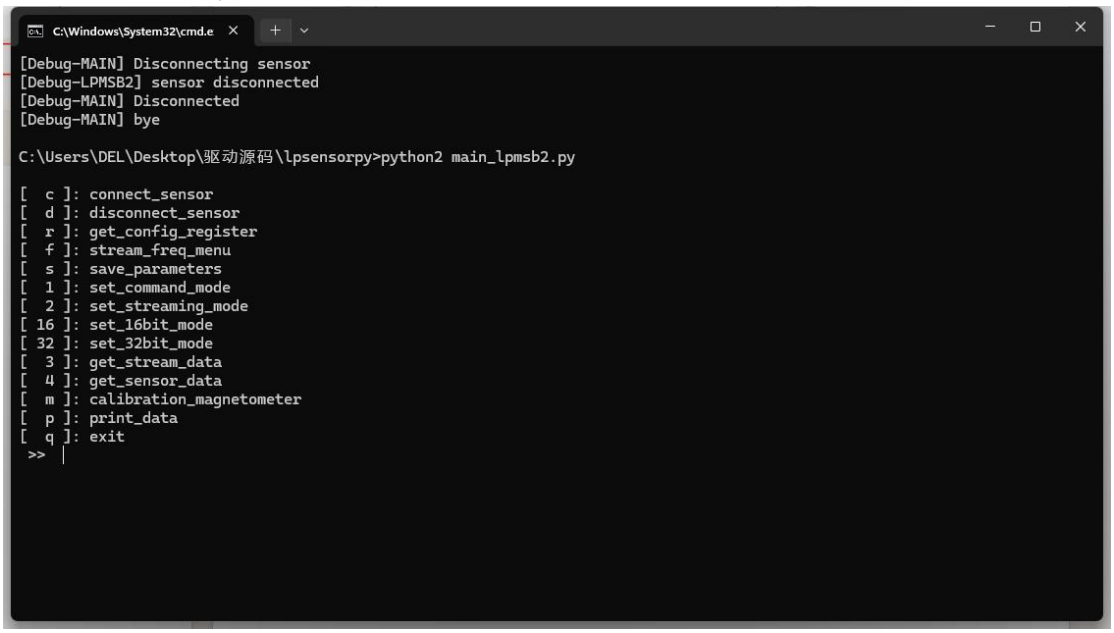


2. 对应端口可通过以下方式查看

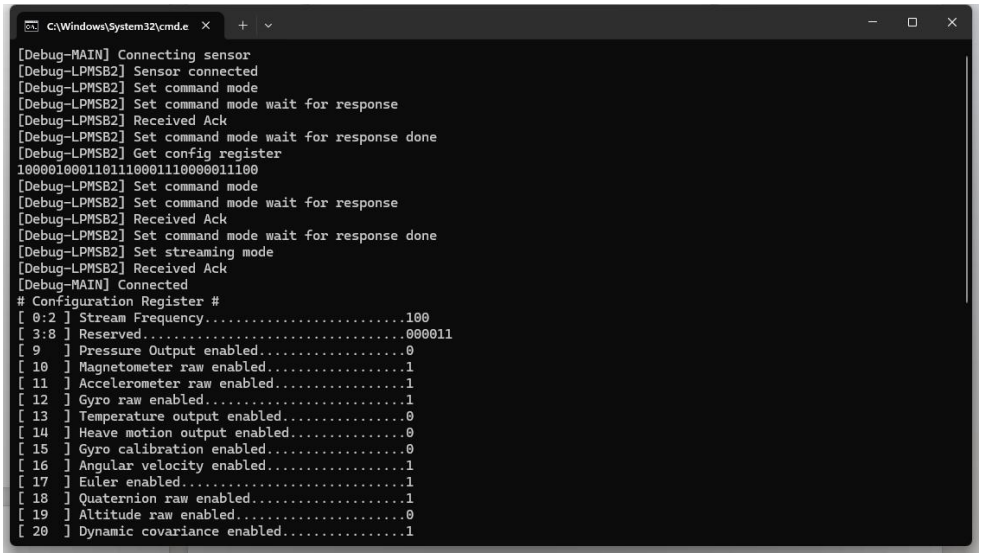




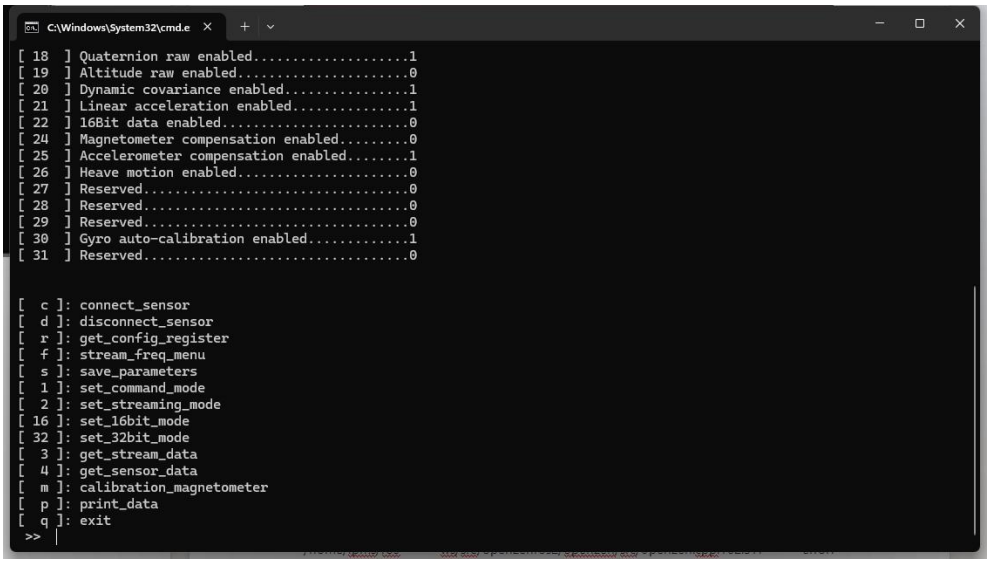
3 运行程序后会出现一下界面



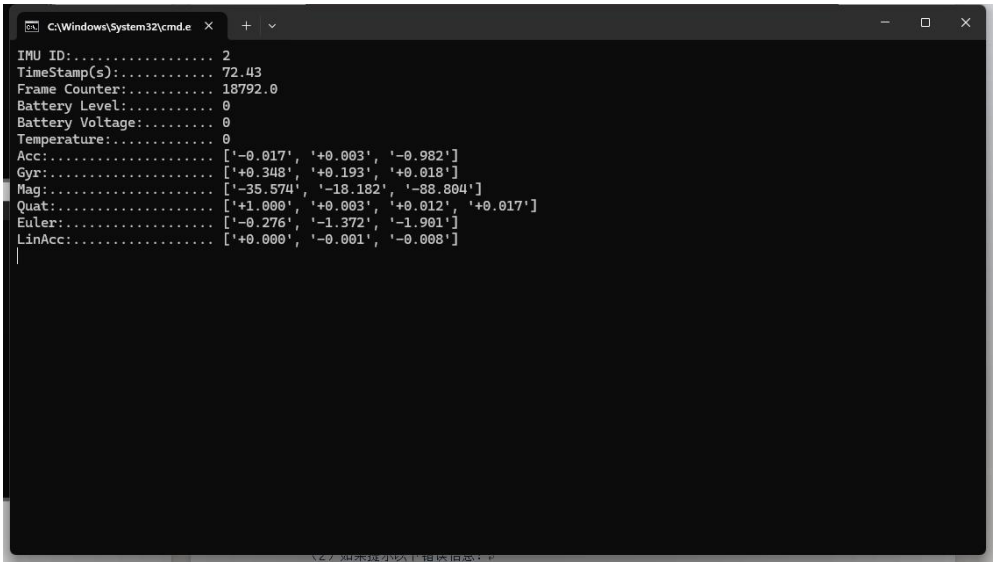
4 输入 c 进行连接 B2



连接后会显示该界面



5 输入 p 打印 B2 数据

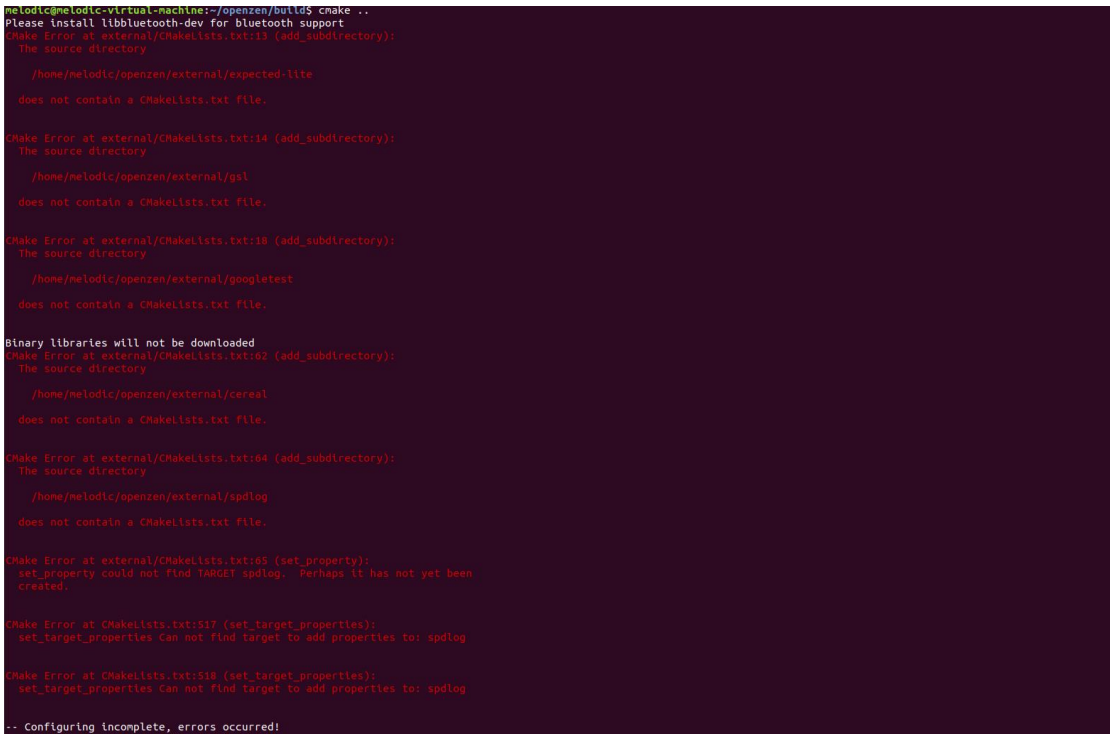


七、常见问题

1 驱动编译和运行

1.Openzen

1. cmake 错误

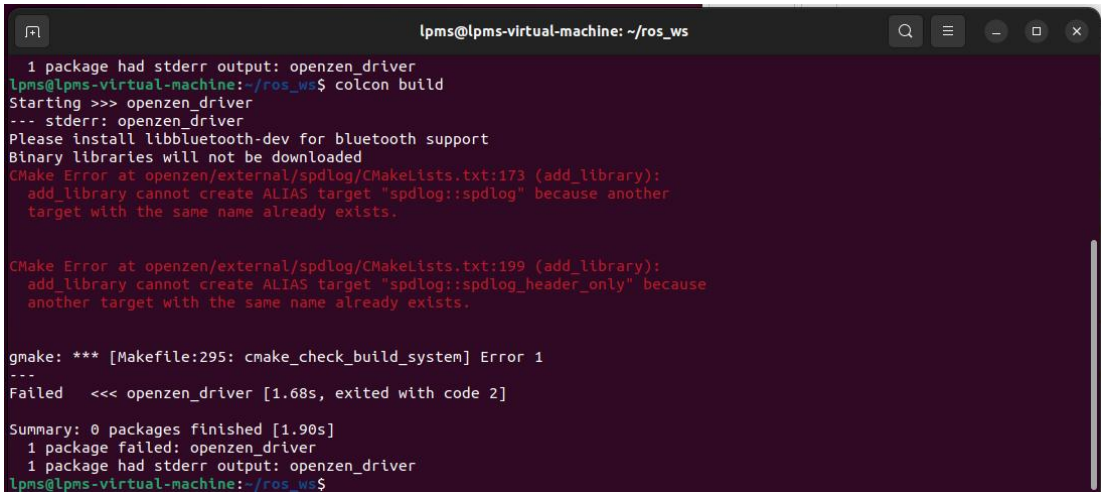


解决方法：
在当前目录进行 git submodule update --init --recursive；
如果需要用到蓝牙的可以安装蓝牙的库 sudo apt-get install libbluetooth-dev。

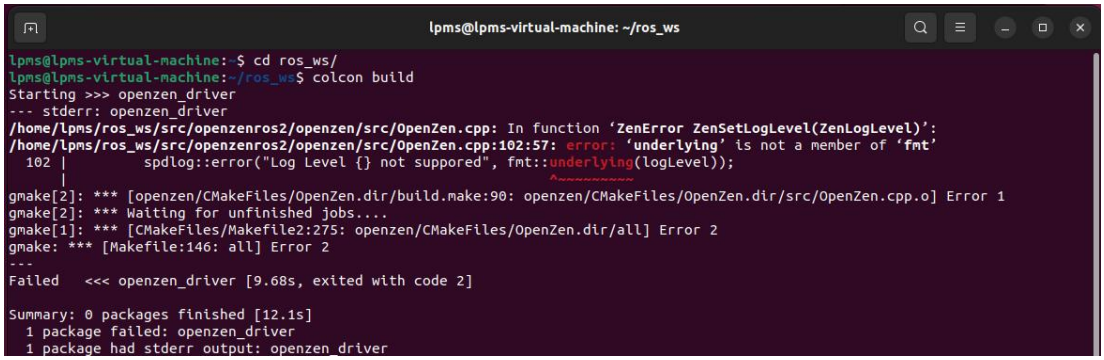
2.OpenzenROS2

1.编译错误

（1）如果提示以下错误信息：

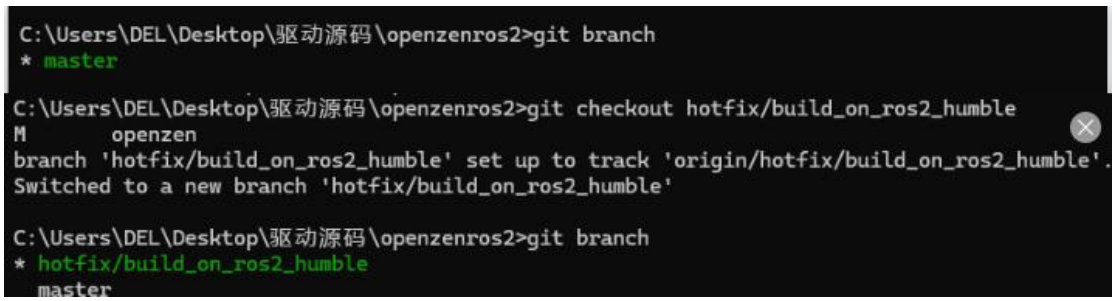


在注释后出现



解决方法：

第一步：修改分支



第二步：

```
lpms@lpms-virtual-machine:~/new ros_ws$ colcon build
Starting >>> openzen_driver
[Processing: openzen_driver]
--- stderr: openzen_driver
Please install libbluetooth-dev for bluetooth support
Binary libraries will not be downloaded
---
Finished <<< openzen_driver [52.3s]

Summary: 1 package finished [52.5s]
  1 package had stderr output: openzen_driver
```

2.如果提示以下错误信息：

```
lpms@lpms-virtual-machine:~/ros_ws$ ros2 run openzen_driver openzen_node --ros-args --remap __ns:=/openzen
Package 'openzen_driver' not found
lpms@lpms-virtual-machine:~/ros_ws$
```

解决方法：通常是因为缺少更新你的工作空间环境步骤导致的，需要刷新环境 source ./install/setup.bash

3.如果提示以下错误信息：

```
lpms@lpms-virtual-machine:~/ros_ws$ ros2 run openzen_driver openzen_node --ros-args --remap __ns:=/openzen -p sensor_name
:="LPMSCU2000573" -p baudrate:=115200
[INFO] [1734419053.689094943] [openzen.openzen_node]: Connecting directly to sensor LPMSCU2000573 over interface LinuxDev
ice
[ERROR] [1734419053.713442533] [openzen.openzen_node]: Cannot connect directly to sensor.  Make sure you have the user ri
ghts to access serial devices.
^C[INFO] [1734419064.648758733] [rclcpp]: signal_handler(signal_value=2)
```

解决方法：

1.检查传感器对应的端口权限是否给上，例子：sudo chmod 777 /dev/ttyUSB0（以实际的端口为准 USB0 只是举例子）；

2.检查是否用户名添加到组，例子：sudo adduser melodic dialout（以实际的用户名为准 melodic 只是举例子可通过 whoami 查看本机的用户名）；

3.检查接线是否正确，查看上位机使用手册的硬件接线进行检查；

4.尝试指定端口和波特率连接传感器，例子：

ros2 run openzen_driver openzen_node --ros-args --remap __ns:=/openzen -p sensor_name:="devicefile:/dev/ttyUSB0" -p baudrate:=921600

5.如果是已经是指定端口和波特率连接传感器了，请检查波特率是否正确和收发是否正常（能收到 IMU 数据不代表正常还需要可以响应指令，即能发能收），

（1）使用 cutecom 之类的串口助手十六进制发送 3A 01 00 06 00 00 00 07 00 0D 0A（命令模式指令）是否会停止数据流输出

（2）发送 3A 01 00 07 00 00 00 08 00 0D 0A 是否会恢复数据流输出（流模式）

如果是 485 通信发送 3A 01 00 06 00 00 00 07 00 0D 0A（命令模式指令）会返回数据包；

如果以上方法均无法解决您的问题，在 windows 上用上位机软件连接传感器查看传感器是否正常；同时需要检查硬件接线和是否支持传感器的波特率。

3.OpenzenROS1

1.如果提示以下错误信息：

```
melodic@melodic-virtual-machine:~$ cd catskin_ws/
melodic@melodic-virtual-machine:~/catskin_ws$ rosrun openzen_sensor openzen_sens
or_node
[rospack] Error: package 'openzen_sensor' not found
melodic@melodic-virtual-machine:~/catskin_ws$
```

解决方法：通常是因为缺少更新你的工作空间环境步骤导致的，需要刷新环境 source ./devel/setup.bash

2.如果提示一下错误信息：

```
melodic@melodic-virtual-machine:~/catskin_ws$ rosrun openzen_sensor openzen_sens
or_node
[ INFO] [1734418038.777258793]: OpenZen sensors will be listed
[ INFO] [1734418038.778080190]: OpenZen sensor with name IG14859003E004A on IO s
ystem LinuxDevice found
[ INFO] [1734418038.778120797]: Connecting to found sensor IG14859003E004A on IO
 system LinuxDevice
[ERROR] [1734418047.288349203]: Cannot connect to sensor found with discovery. M
ake sure you have the user rights to access serial devices.
[ERROR] [1734418047.288402034]: OpenZen sensor could not be connected
melodic@melodic-virtual-machine:~/catskin_ws$
```

解决方法：

1.检查传感器对应的端口权限是否给上，例子：sudo chmod 777 /dev/ttyUSB0（以实际的端口为准 USB0 只是举例子）；

2.检查是否用户名添加到组，例子：sudo adduser melodic dialout（以实际的用户名为准 melodic 只是举例子可通过 whoami 查看本机的用户名）；

3.检查接线是否正确，查看上位机使用手册的硬件接线进行检查；

4.尝试指定端口和波特率连接传感器，例子：

```
rosrun openzen_sensor openzen_sensor_node_sensor_name
:="devicefile:/dev/ttyUSB0" _baudrate:=921600
```

5.如果是已经是指定端口和波特率连接传感器了，请检查波特率是否正确和收发是否正常（能收到 IMU 数据不代表正常还需要可以响应指令，即能发能收），

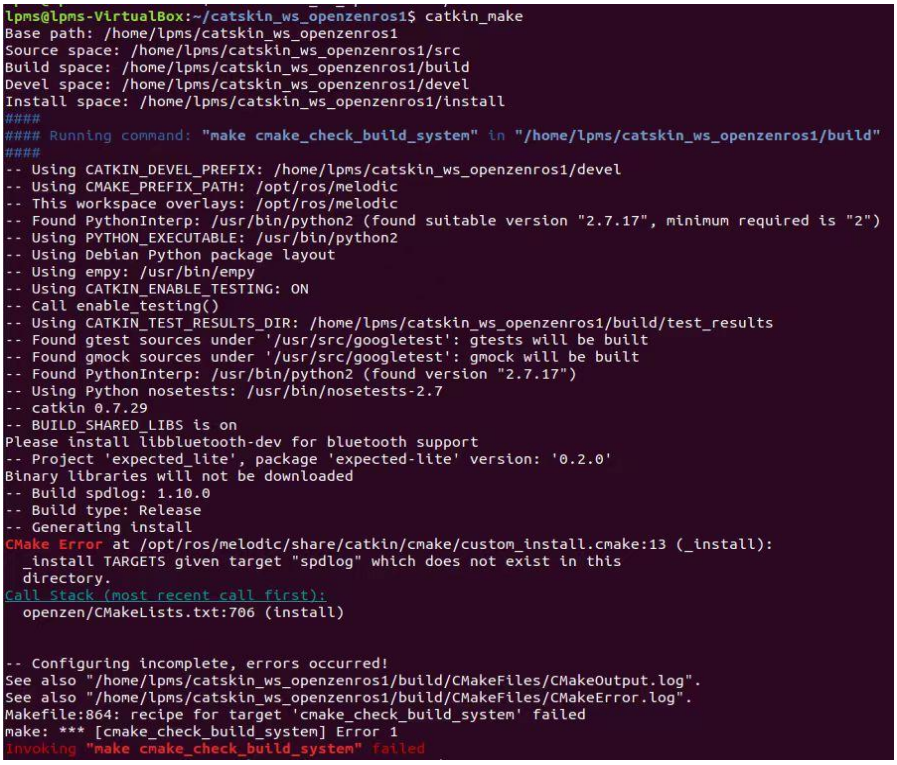
（1）使用 cutecom 之类的串口助手十六进制发送 3A 01 00 06 00 00 00 07 00 0D 0A（命令模式指令）是否会停止数据流输出

（2）发送 3A 01 00 07 00 00 00 08 00 0D 0A 是否会恢复数据流输出（流模式）

如果是 485 通信发送 3A 01 00 06 00 00 00 07 00 0D 0A（命令模式指令）会返回数据包；

如果以上方法均无法解决您的问题，在 windows 上用上位机软件连接传感器查看传感器是否正常；同时需要检查硬件接线和是否支持传感器的波特率。

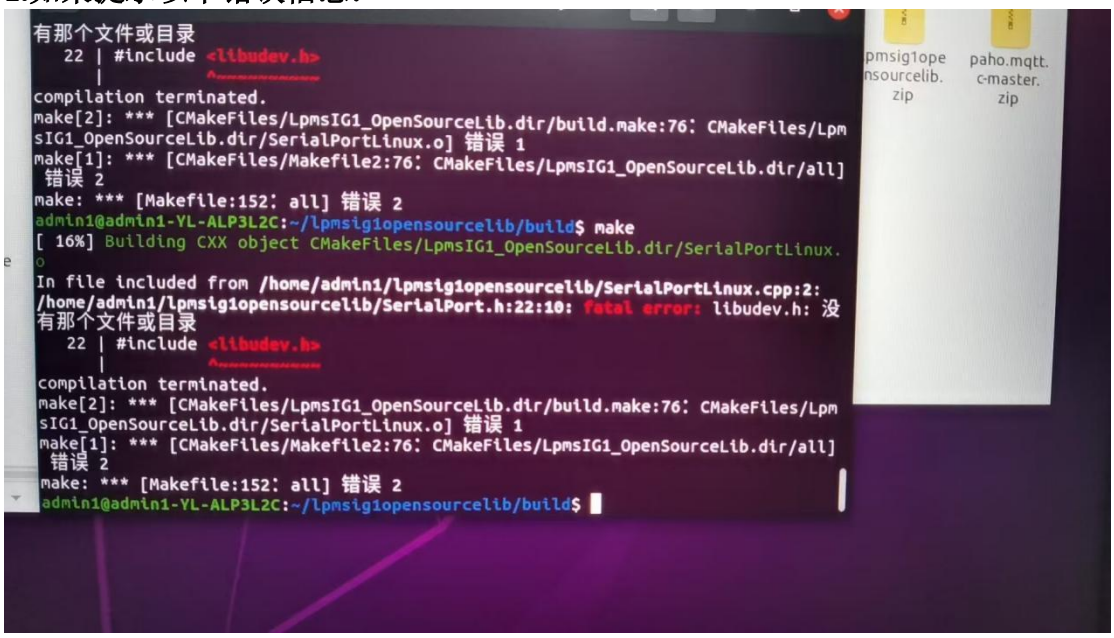
3.如果提示以下错误信息：



解决方法：将 cmake 版本更新为最新即可

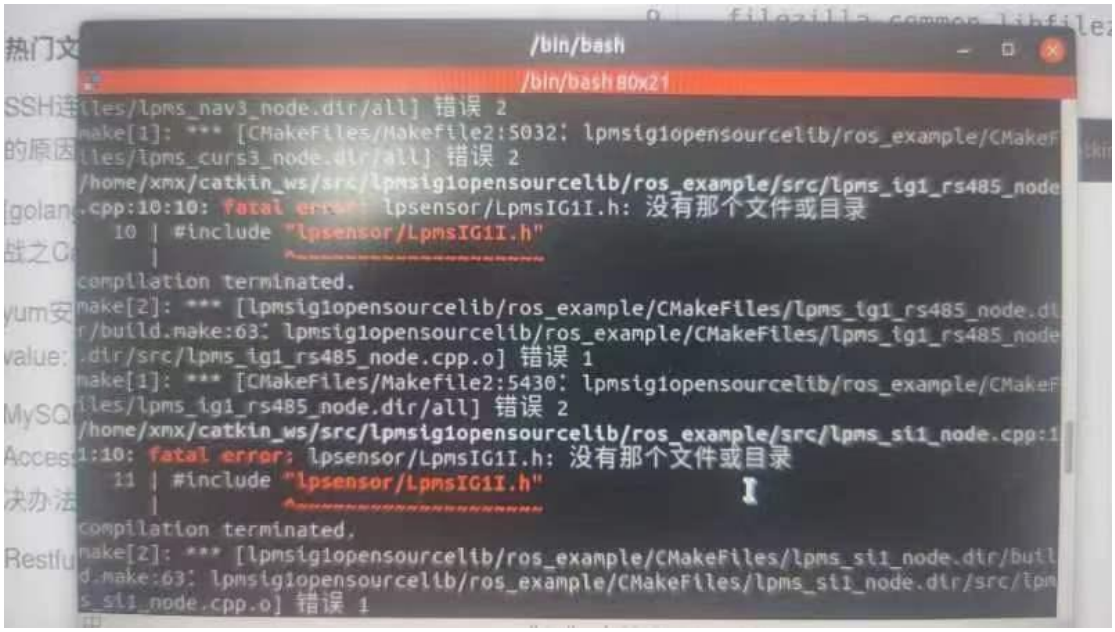
4.LpmsIG1OpenSource/LpmsIG1NAV2OpenSource

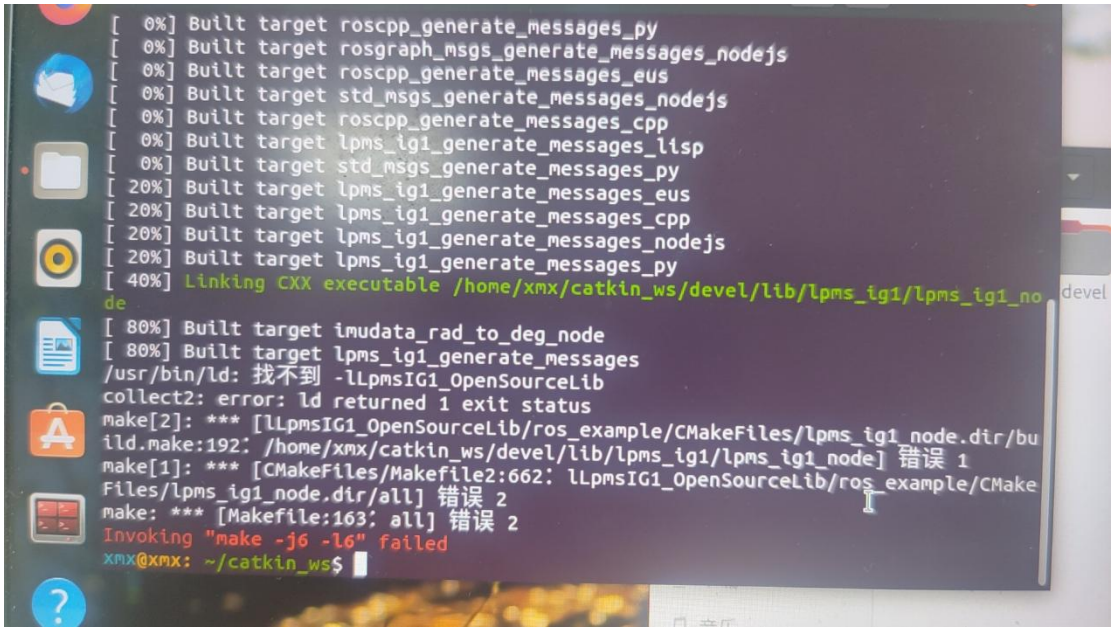
1.如果提示以下错误信息：



解决方法：sudo apt-get install libudev-dev

2.如果提示以下错误信息：





解决方法：sudo dpkg -i libLpmsIG1_OpenSource-x.y.z-Linux.deb(其中 xyz 是 make package 下载下来的安装包版本号请对应版本比如版本号是 0.3.3xyz 分别填写 0 3 3)

3.如果提示以下错误信息：

dpkg: 错误: 无法访问归档 'libLpmsIG1_OpenSource-x.y.z-Linux.deb': 没有那个文件或目录

解决方法：sudo dpkg -i libLpmsIG1_OpenSource-x.y.z-Linux.deb(其中 xyz 是 make package 下载下来的安装包版本号请对应版本比如版本号是 0.3.3xyz 分别填写 0 3 3)

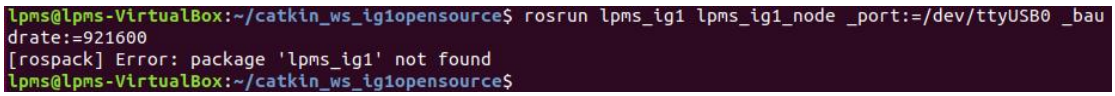
4.如果提示以下信息：

找不到-lLpmsIG_OopensourceLib

collcet2：error:ld returned 1exit status

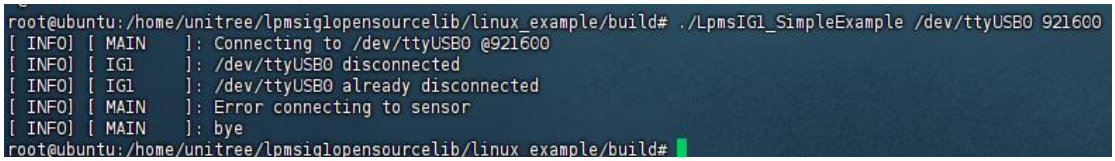
解决方法：sudo dpkg -i libLpmsIG1_OpenSource-x.y.z-Linux.deb(其中 xyz 是 make package 下载下来的安装包版本号请对应版本比如版本号是 0.3.3，xyz 分别填写 0 3 3)

5.如果提示以下信息：

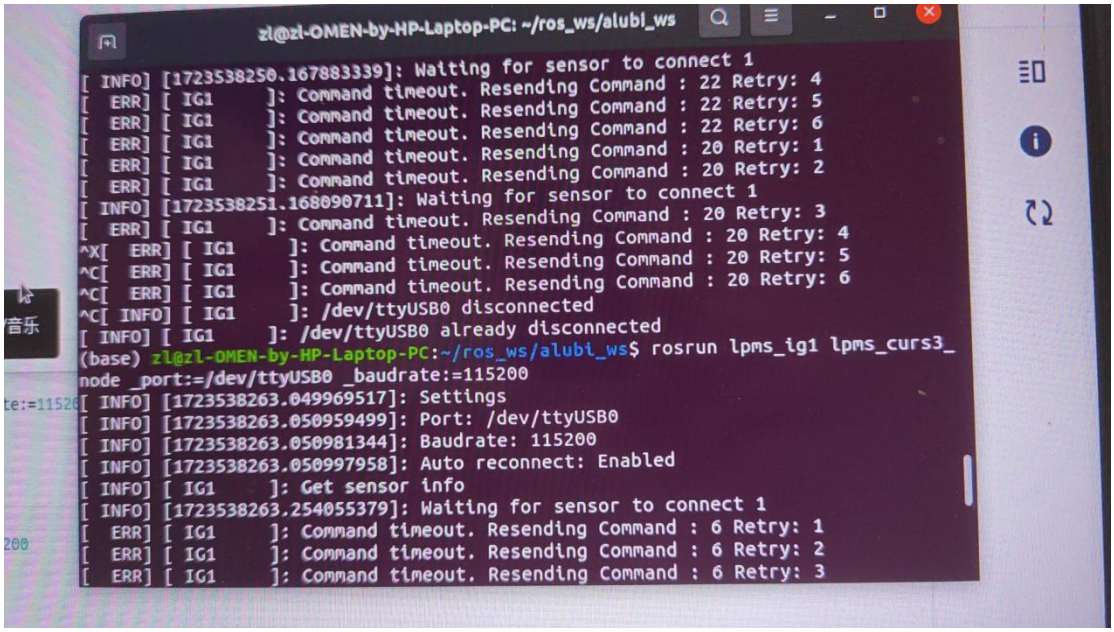


解决方法：source ./devel/setup.bash

6.连接 linux 历程出现以下信息：



或者连接 ros 历程出现类似一下信息：



解决方法：

1.检查传感器对应的端口权限是否给上，例子：sudo chmod 777 /dev/ttyUSB0（以实际的端口为准 USB0 只是举例子）；

2.检查是否用户名添加到组，例子：sudo adduser melodic dialout（以实际的用户名为准 melodic 只是举例子可通过 whoami 查看本机的用户名）；

3.检查接线是否正确，查看上位机使用手册的硬件接线进行检查；

4.尝试指定端口和波特率连接传感器，例子：

rosrun openzen_sensor openzen_sensor_node _sensor_name:="devicefile:/dev/ttyUSB0" _baudrate:=921600

5.如果是已经是指定端口和波特率连接传感器了，请检查波特率是否正确和收发是否正常，使用 cutecom 之类的串口助手十六进制发送 3A 01 00 06 00 00 00 07 00 0D 0A（命令模式指令）是否会停止数据流输出，发送 3A 01 00 07 00 00 00 08 00 0D 0A 是否会恢复数据流输出（流模式），如果是 485 通信发送 3A 01 00 06 00 00 00 07 00 0D 0A（命令模式指令）会返回数据包；

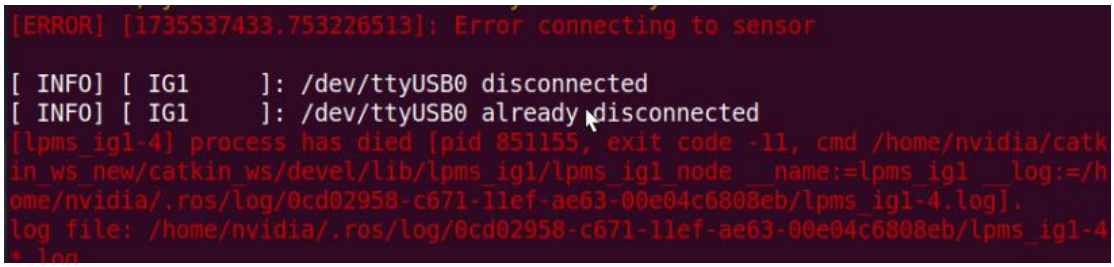
如果以上方法均无法解决您的问题可以查看上位机使用手册，在 windows 上用上位机软件连接传感器查看传感器是否正常；同时需要检查硬件是否支持传感器的波特率。

7.为什么运行 node 节点可以正常运行而运行 launch 文件就连接不上

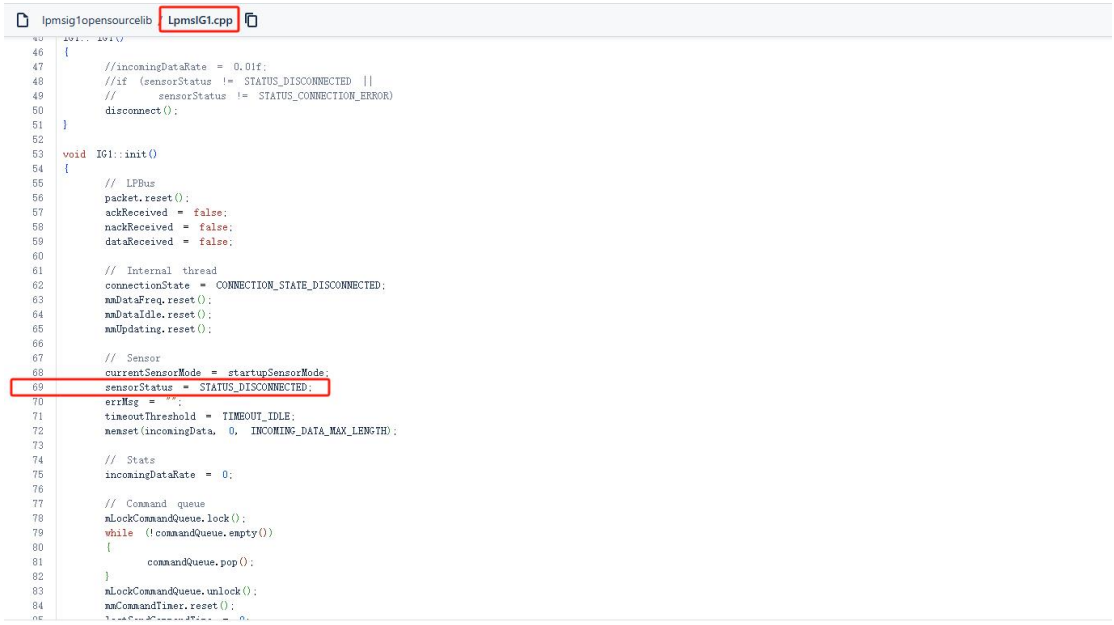
解决方法：麻烦请检查 launch 文件中的端口和波特率是否正确（如下图），如果有错修改为正确的波特率和端口重新编译即可。



8. 断开节点在启动时偶尔出现报错的情况



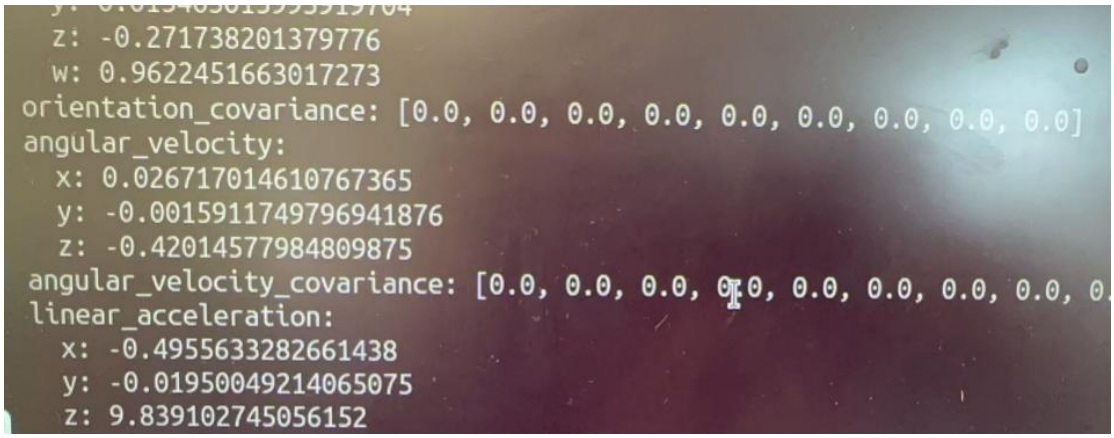
解决方法：将 LpmsIG1.cpp 里面的第 69 行代码进行注释即可（如下图）



2 驱动输出的数值

1.为什么驱动输出加速度数据为 9.8（openzenRos1、openzenRos2 和 IG1opensource 同理）

答：以 Lpmsig1opensourcelib 驱动为例子



（1）驱动的 linear_acceleration 线性加速度这一栏，是由 accCalibrated 经过校准的加速度来输出的如下图，IMU 输出的加速度 accCalibrated 是重力加速度 z 轴朝上是数值一般为 1 或-1。

```
// Fill orientation quaternion
imu_msg.orientation.w = sd.quaternion.data[0];
imu_msg.orientation.x = -sd.quaternion.data[1];
imu_msg.orientation.y = -sd.quaternion.data[2];
imu_msg.orientation.z = -sd.quaternion.data[3];

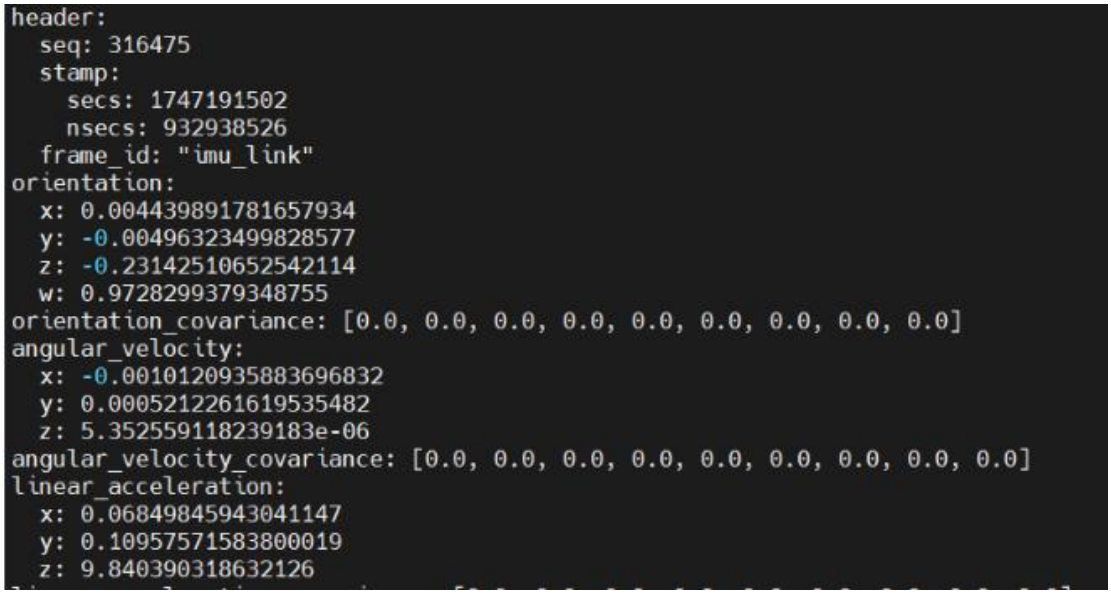
// Fill angular velocity data
// - scale from deg/s to rad/s
imu_msg.angular_velocity.x = sd.gyroIAlignmentCalibrated.data[0]*3.1415926/180;
imu_msg.angular_velocity.y = sd.gyroIAlignmentCalibrated.data[1]*3.1415926/180;
imu_msg.angular_velocity.z = sd.gyroIAlignmentCalibrated.data[2]*3.1415926/180;

// Fill linear acceleration data
imu_msg.linear_acceleration.x = -sd.accCalibrated.data[0]*9.81;
imu_msg.linear_acceleration.y = -sd.accCalibrated.data[1]*9.81;
imu_msg.linear_acceleration.z = -sd.accCalibrated.data[2]*9.81;
```

（2）IMU 静止正方在平方在水平面时，上位机上加速度的数值显示为 0，0，1 分别代表 X Y Z，其中 1 代表一个 G 的重力加速度，在此基础上，驱动代码将 X，Y，Z 分别乘以标准大气压下的重力加速度 9.81g，便得到了驱动上显示的 0，0，9.81；

（3）想要 linear_acceleration 线加速度由线加速度来进行输出可以将 accCalibrated 改为 linAcc;

例如：imu_msg.linear_acceleration.z = -sd.linAcc.data[2]*9.81;

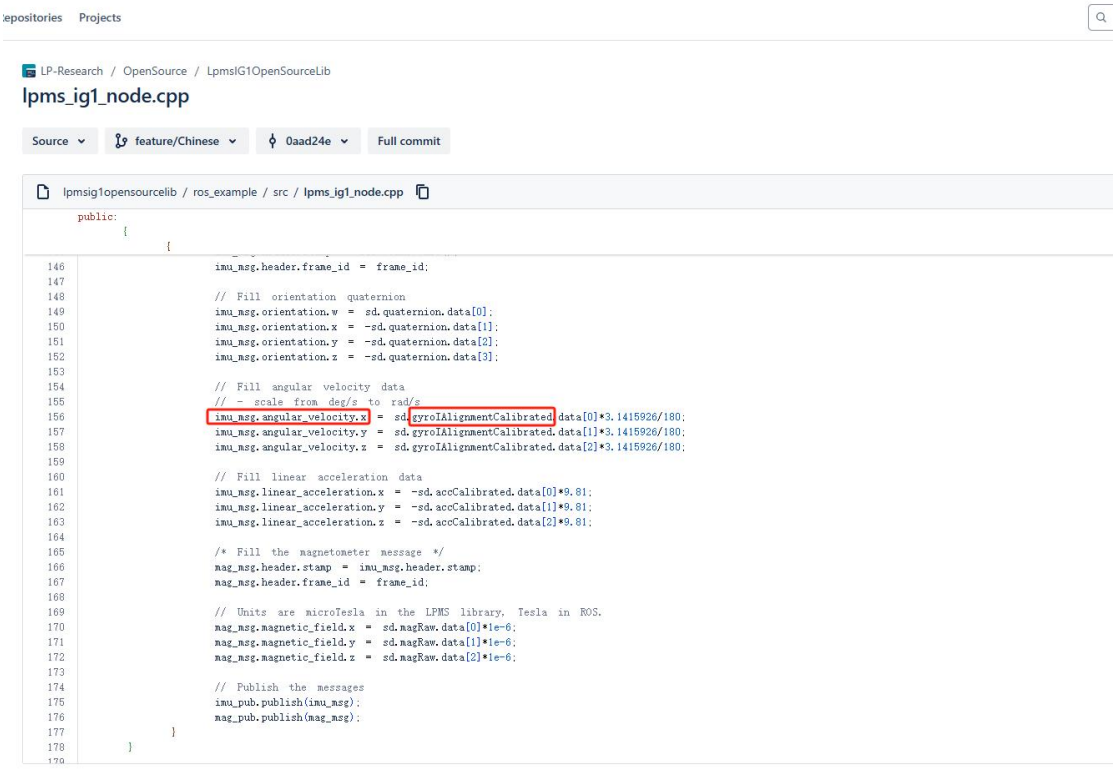


在 x 或 y 轴出现数值有 0.1 的情况认为也是正常的，因为驱动是将每一个重力加速度都*9.81 的数值，如果将 9.81 去除了之后就 y 轴而言重力加速度也就只有 0.01；

2.为什么发现连接上 Linux 驱动或 ROS 驱动后发现输出的数据为 0

解决方法：

先查看相对应的传感器节点输出的数据来自传感器的哪一个数据，例如下图



在上位机上勾选上对应的数据 gyroIAlignmentCalibrated 点击应用输出并保存（保存操作请查看上位机使用手册中的常见问题里面有保存操作），再连接上驱动后便可输出数据；

3.如果想要修改输出的内容可以在对应传感器的节点上进行修改

解决方法：

LP-Research / OpenSource / LpmsIG1OpenSourceLib

lpms_ig1_node.cpp

Source | feature/Chinese | 0aad24e | Full commit

lpmsig1opensourcelib / ros_example / src / lpms_ig1_node.cpp

```
public:
        {
                {
                        imu_msg.header.frame_id = frame_id;

                        // Fill orientation quaternion
                        imu_msg.orientation.w = sd.quaternion.data[0];
                        imu_msg.orientation.x = -sd.quaternion.data[1];
                        imu_msg.orientation.y = -sd.quaternion.data[2];
                        imu_msg.orientation.z = -sd.quaternion.data[3];

                        // Fill angular velocity data
                        //   scale from deg/s to rad/s
                        imu_msg.angular_velocity.x = sd.gyroIAlignmentCalibrated.data[0]*3.1415926/180;
                        imu_msg.angular_velocity.y = sd.gyroIAlignmentCalibrated.data[1]*3.1415926/180;
                        imu_msg.angular_velocity.z = sd.gyroIAlignmentCalibrated.data[2]*3.1415926/180;

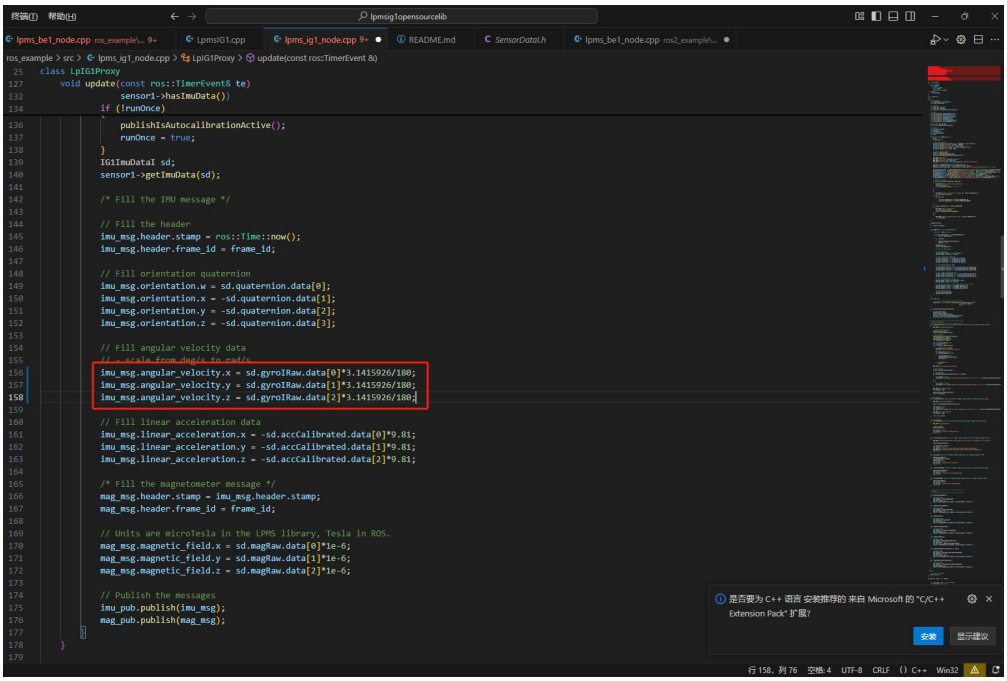
                        // Fill linear acceleration data
                        imu_msg.linear_acceleration.x = -sd.accCalibrated.data[0]*9.81;
                        imu_msg.linear_acceleration.y = -sd.accCalibrated.data[1]*9.81;
                        imu_msg.linear_acceleration.z = -sd.accCalibrated.data[2]*9.81;

                        /* Fill the magnetometer message */
                        mag_msg.header.stamp = imu_msg.header.stamp;
                        mag_msg.header.frame_id = frame_id;

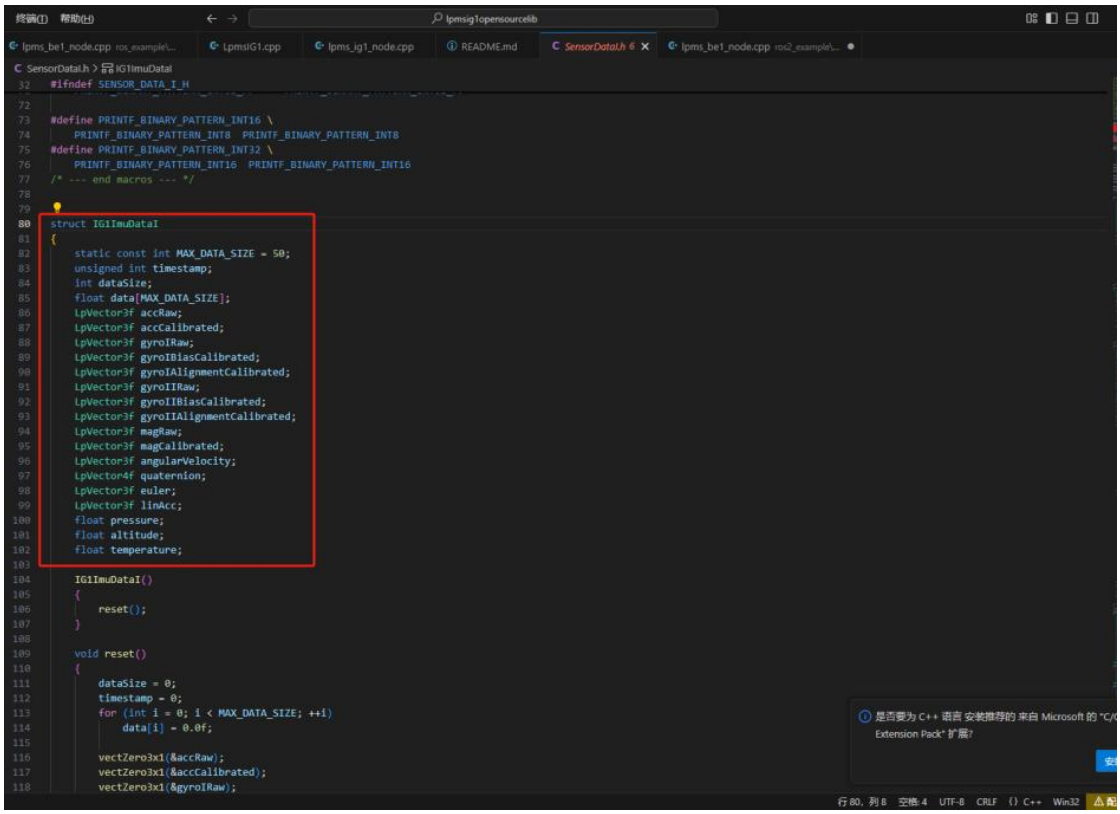
                        // Units are microTesla in the LPMS library, Tesla in ROS.
                        mag_msg.magnetic_field.x = sd.magRaw.data[0]*1e-6;
                        mag_msg.magnetic_field.y = sd.magRaw.data[1]*1e-6;
                        mag_msg.magnetic_field.z = sd.magRaw.data[2]*1e-6;

                        // Publish the messages
                        imu_pub.publish(imu_msg);
                        mag_pub.publish(mag_msg);
                }
        }
```

imu_msg.angular_velocity.x = sd.gyroIAlignmentCalibrated.data[0]*3.1415926/180，比如想要原始的陀螺仪数据就把语句中的 gyroIAlignmentCalibrated 修改为 gyroIRaw 即可；



想要知道有哪一些数据可以修改可以查看 SensorDataI.h 中的 IG1ImuDataI 的构造函数，



里面有传感器可以输出的所有数据（数据的解释请查看上位机使用手册）。

注意：有些传感器只有陀螺仪 1，有些传感器只有陀螺仪 2 需要结合传感器的实际数据进行修改

4 发现修改了 rate 值后通过 rostopic hz 打印出来的 hz 并不是 rate 对应值

答：需要注意（1）使用 IG1opensource 想提高 rostopic hz 的频率单独修改代码中的 rate 值是不可以的，rate 值得修改只影响 rostopic hz 的最大值；
（2）使用 openzenRos1 或者 openzenRos2 这两个驱动 hz 频率是由 IMU 决定的，需要修改 IMU 的输出频率）

IG1opensource：

lpmsnav2opensourcelib / ros_example / src / lpms_nav_node.cpp

```
{
public:
    int  rate;
    double  timeout_threshold;
    int  type;
    bool  quit;

    LpNavProxy(ros::NodeHandle  h)  :
        nh(h),
        private_nh("~")
    {
        // Get  node  parameters
        private_nh.param<std::string>("port",  comportNo,  "/dev/ttyUSB0");
        private_nh.param("baudrate",  baudrate,  115200);
        private_nh.param("autoreconnect",  autoReconnect,  true);
        private_nh.param<std::string>("frame_id",  frame_id,  "imu");
        private_nh.param("rate",  rate,  200);
        private_nh.param("timeout_threshold",  timeout_threshold,  5.0);

        // Create  LpmsNAV  object
        sensor1  =  LpmsNAVFactory(comportNo,  baudrate);
```

情况一：例如已经设置了 rate 值为 250，但是 rostopic hz 的频率依旧为 100hz（传感器出厂默认 hz）
1. 用上位机软件检查传感器的输出频率是否为对应 250hz，
2. 如果不是 250hz，需要将传感器的 hz 修改 250，然后点击保存数据并将传感器断电重启（必须执行不然无法相应设置）

情况二：已经将传感器设置为对应的 250hz，rate 值也设置为了 250，但是 rostopic hz 的频率只有 200 左右不足 250
1. 通常是由于输出的数据量过大导致输出的频率降低（115200 波特率 100hz 的情况下最高可以输出 7 组 imu 数据，传感器输出了多少组数据请在上位机上查看）
2. 可以尝试两个方式：
   （1）提高波特率，在上位机软件上进行设置波特率，选择完波特率后点击保存数据并将传感器断电重启（必须执行不然无法相应设置）
   （2）降低数据输出的数量，在上位机软件上取消勾选不需要的 imu 数据并点击应用，选择完波特率后点击保存数据并将传感器断电重启（必须执行不然无法相应设置）。

openzenRos1 或者 openzenRos2
情况一：连接上驱动后，但是 rostopic hz 的频率为 100hz（传感器出厂默认 hz）；

1. 用上位机软件检查传感器的输出频率是否为对应 100hz，
2. 如果是 100hz，需要将传感器的 hz 修改 250，然后点击保存数据并将传感器断电重启（必须执行不然无法相应设置）
3. 再次连接驱动

情况二：已经将传感器设置为对应的 250hz，rate 值也设置为了 250，但是 rostopic hz 的频率只有 200 左右不足 250；
1. 通常是由于输出的数据量过大导致输出的频率降低（115200 波特率 100hz 的情况下最高可以输出 7 组 imu 数据，传感器输出了多少组数据请在上位机上查看）
2. 可以尝试两个方式：
（1）提高波特率，在上位机软件上进行设置波特率，选择完波特率后点击保存数据并将传感器断电重启（必须执行不然无法相应设置）
（2）降低数据输出的数量，在上位机软件上取消勾选不需要的 imu 数据并点击应用，选择完波特率后点击保存数据并将传感器断电重启（必须执行不然无法相应设置）
（3）再次连接驱动